RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: YOGITHA.S

Academic Year: 2026-27 **Semester:** III **Branch:** BE.CSE.

Register No. 2116250701865

Certified that this is the bonafide record of work done by the above student in the CS23334 - Fundamentals of Data Science *Laboratory during the academic year 2026 – 2027.*

Signature of Faculty in-charge

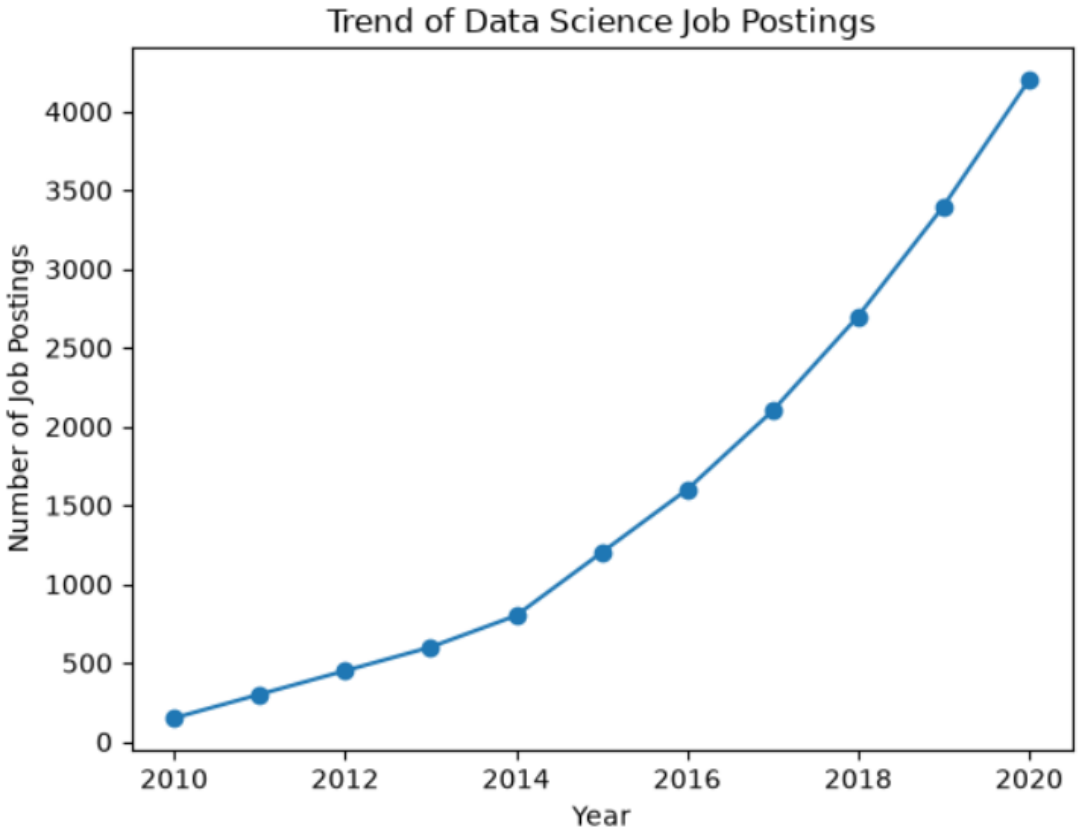
Submitted for the Practical Examination held on 26.11.2026.

Internal Examiner

External Examiner

```python
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```




```python
import pandas as pd
import matplotlib.pyplot as mp
data={'Job role':['Data Analyst','Data Scientist','Mechine Learning Engineer','Data Engineer','Data Architect','Business Analyst'],
      'Count':[120,200,180,140,90,70]
      }
df=pd.DataFrame(data)
print(df)
mp.figure(figsize=(9,6))
mp.title('Various Data science Roles')
mp.xlabel('Job role')
mp.ylabel('Number of employees')
mp.xticks(rotation=15)
mp.bar(df['Job role'],df['Count'])
mp.show()
```


Various Data science Roles
Number of employees
0
25
50
75
100
125
150
175
200
Data Analyst
Data Scientist
Mechine Learning Engineer
Data Engineer
Data Architect
Business Analyst
Job role

```python
#Structured data
import pandas as pd
data={
    'Name':['Yamuna','Yuga','Yogitha','Merylin'],
    'Age':[20,21,18,19],
    'ID':[201,208,209,210]
}
df=pd.DataFrame(data)
print("Structured data")
print(df)

#Semi Structured data
import json
data={
    'Name':'Bob',
    'Age':20,
    'Id':201,
    'Address':'Chennai'
}
print("Semi Structured Data")
print(json.dumps(data,indent=5))

#Unstructured data
text="""
JSON stands for JavaScript Object Notation.
It organizes data using key-value pairs,
but it is not arranged in rows and columns like a table.
"""
print("Unstructured data")
print(text)
```

```
Structured data
      Name  Age   ID
0   Yamuna   20  201
1     Yuga   21  208
2  Yogitha   18  209
3  Merylin   19  210
Semi Structured Data
{
     "Name": "Bob",
     "Age": 20,
     "Id": 201,
     "Address": "Chennai"
}
Unstructured data

JSON stands for JavaScript Object Notation.
It organizes data using key-value pairs,
but it is not arranged in rows and columns like a table.
```

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher suite = Fernet(key)
plain text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
decrypted text = cipher suite.decrypt(cipher text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted text)
```

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqcsNM0M/mJCO6ByLgOvvnhtlRPL0PNmeV/92N2bKycuSuYpPJ1vttHix4xIMLQ5iFbv4cJxDbFln9mh 4RFstULCJDvfJsIIY1x w/iUBAB5NXaI1QNladL0PFJuOj
MkAbkw9'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

file_path = r'C:\sales_data.csv'
df = pd.read_csv(r"C:\Users\kyoga\Downloads\sales_data - Sheet1.csv")

print(df.head())

print(df.isnull().sum())

df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

print(df.describe())

product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()

print(product_summary)

plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
sales_over_time = df.groupby('Date').agg({
    'Sales': 'sum'
}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()

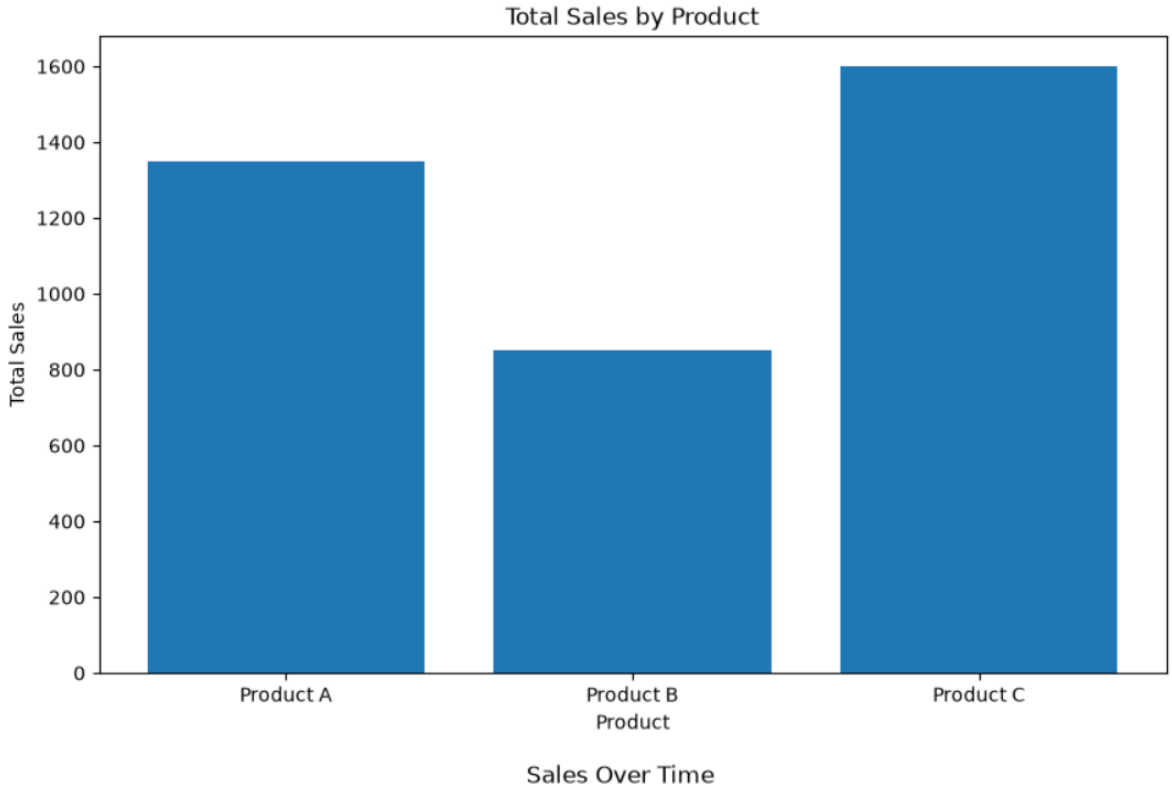
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc=np.sum,
    fill_value=0
)

print(pivot_table)

correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

```
         Date     Product  Sales  Quantity Region
0  01-01-2023   Product A    200         4  North
1  02-01-2023   Product B    150         3  South
2  03-01-2023   Product A    220         5  North
3  04-01-2023   Product C    300         6   East
4  05-01-2023   Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
              Sales    Quantity
count    16.000000   16.000000
mean    237.500000    5.375000
std      64.031242    1.746425
min     150.000000    3.000000
25%     187.500000    4.000000
50%     225.000000    5.500000
75%     302.500000    7.000000
max     340.000000    8.000000
      Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```



Sales Over Time

```
import numpy as np
import pandas as pd

df=pd.read_csv(r"C:\Users\kyoga\Downloads\Hotel_Dataset - Hotel_Dataset.csv")
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

```
df.duplicated()
```

```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   CustomerID        11 non-null     int64 
 1   Age_Group         11 non-null     str   
 2   Rating(1-5)       11 non-null     int64 
 3   Hotel             11 non-null     str   
 4   FoodPreference    11 non-null     str   
 5   Bill              11 non-null     int64 
 6   NoOfPax           11 non-null     int64 
 7   EstimatedSalary   11 non-null     int64 
 8   Age_Group.1       11 non-null     str   
dtypes: int64(5), str(4)
memory usage: 924.0 bytes
```

```
df.drop_duplicates(inplace=True)
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

```
len(df)
```

```
10
```

```
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
index
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+
5	6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
7	8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777	30-35

```
df.drop(['Age_Group.1'],axis=1,inplace=True)
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1	20-25	4	Ibis	veg	1300	2	40000
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000
2	3	25-30	6	RedFox	Veg	1322	2	30000
3	4	20-25	-1	LemonTree	Veg	1234	2	120000
4	5	35+	3	Ibis	Vegetarian	989	2	45000
5	6	35+	3	Ibys	Non-Veg	1909	2	122220
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122
7	8	20-25	7	LemonTree	Veg	2999	-10	345673
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777

```
df.CustomerID.loc[df.CustomerID<0]=np.nan
df.Bill.loc[df.Bill<0]=np.nan
df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
df
```

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary
0	1	20-25	4	Ibis	veg	1300	2	40000
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000
2	3	25-30	6	RedFox	Veg	1322	2	30000
3	4	20-25	-1	LemonTree	Veg	1234	2	120000
4	5	35+	3	Ibis	Vegetarian	989	2	45000
5	6	35+	3	Ibys	Non-Veg	1909	2	122220
6	7	35+	4	RedFox	Vegetarian	1000	-1	21122
7	8	20-25	7	LemonTree	Veg	2999	-10	345673
8	9	25-30	2	Ibis	Non-Veg	3456	3	-99999
9	10	30-35	5	RedFox	non-Veg	-6755	4	87777

```
df.Age_Group.unique()
df.Hotel.unique()
df.FoodPreference.unique
```

```
<bound method Series.unique of 0           veg
1       Non-Veg
2           Veg
3           Veg
4    Vegetarian
5       Non-Veg
6    Vegetarian
7           Veg
8       Non-Veg
9       non-Veg
Name: FoodPreference, dtype: str>
```

```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\kyoga\Downloads\pre_process_datasample - pre_process_datasample - pre_process_datasample - pre_pr.csv")
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
df.info()
df.Country.mode()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     str
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 452.0 bytes
0    France
Name: Country, dtype: str
```

```
df.Country.mode()[0]

type(df.Country.mode())
```

```
pandas.Series
```

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
df
```

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
df
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
pd.get_dummies(df.Country)
```

	France	Germany	Spain
0	True	False	False
1	False	False	True
2	False	True	False
3	False	False	True
4	False	True	False
5	True	False	False
6	False	False	True
7	True	False	False
8	False	True	False
9	True	False	False

```
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],axis=1)
updated_dataset
```

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	No
1	False	False	True	27.0	48000.0	Yes
2	False	True	False	30.0	54000.0	No
3	False	False	True	38.0	61000.0	No
4	False	True	False	40.0	NaN	Yes
5	True	False	False	35.0	58000.0	Yes
6	False	False	True	NaN	52000.0	No
7	True	False	False	48.0	79000.0	Yes
8	False	True	False	50.0	83000.0	No
9	True	False	False	37.0	67000.0	Yes

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     str
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 452.0 bytes
```

```
updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
```

```
0    0
1    1
2    0
3    0
4    1
5    1
6    0
7    1
8    0
9    1
Name: Purchased, dtype: object
```

```
updated_dataset
```

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44.0	72000.0	No
1	False	False	True	27.0	48000.0	Yes
2	False	True	False	30.0	54000.0	No
3	False	False	True	38.0	61000.0	No
4	False	True	False	40.0	NaN	Yes
5	True	False	False	35.0	58000.0	Yes
6	False	False	True	NaN	52000.0	No
7	True	False	False	48.0	79000.0	Yes
8	False	True	False	50.0	83000.0	No
9	True	False	False	37.0	67000.0	Yes

```python
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

array = np.random.randint(1, 100, 16)
print(array)

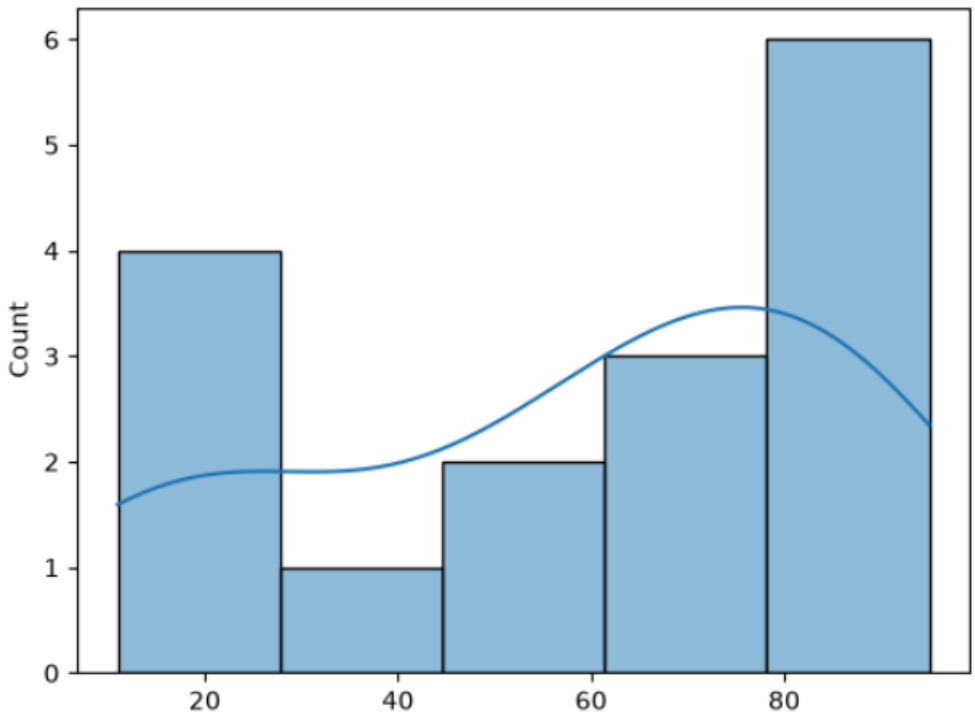
print(array.mean())
print(np.percentile(array, 25))
print(np.percentile(array, 50))
print(np.percentile(array, 75))
print(np.percentile(array, 100))
```

```
[63 42 19 84 20 51 95 58 67 87 70 80 81 87 11 19]
58.375
36.5
65.0
81.75
95.0
```

```python
def outDetection(array):
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)
print(lr, ur)
sns.histplot(array, kde=True)
plt.show()
```

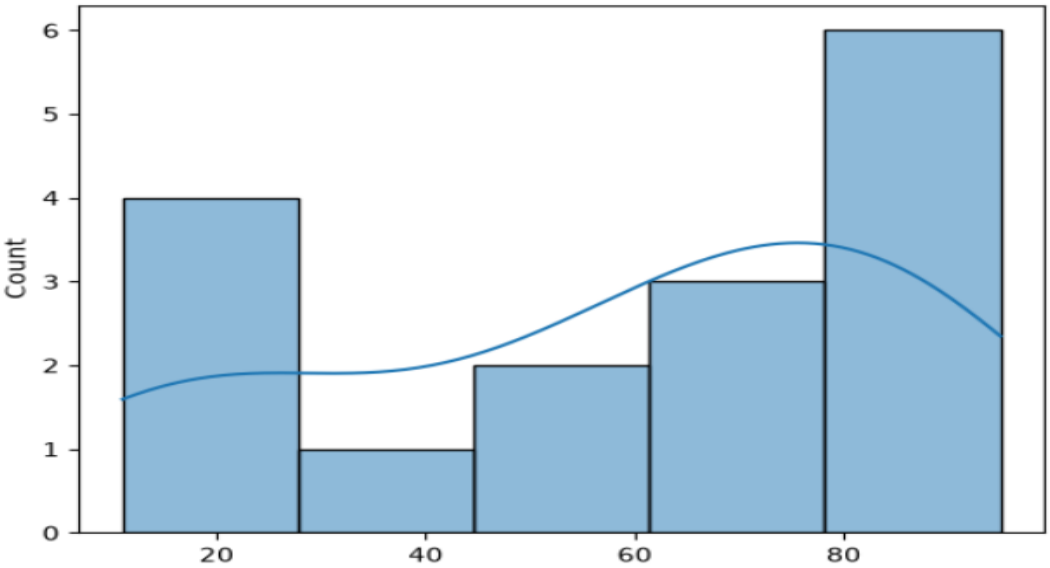
```
-31.375 149.625
```



```
new_array = array[(array > lr) & (array < ur)]
print(new_array)

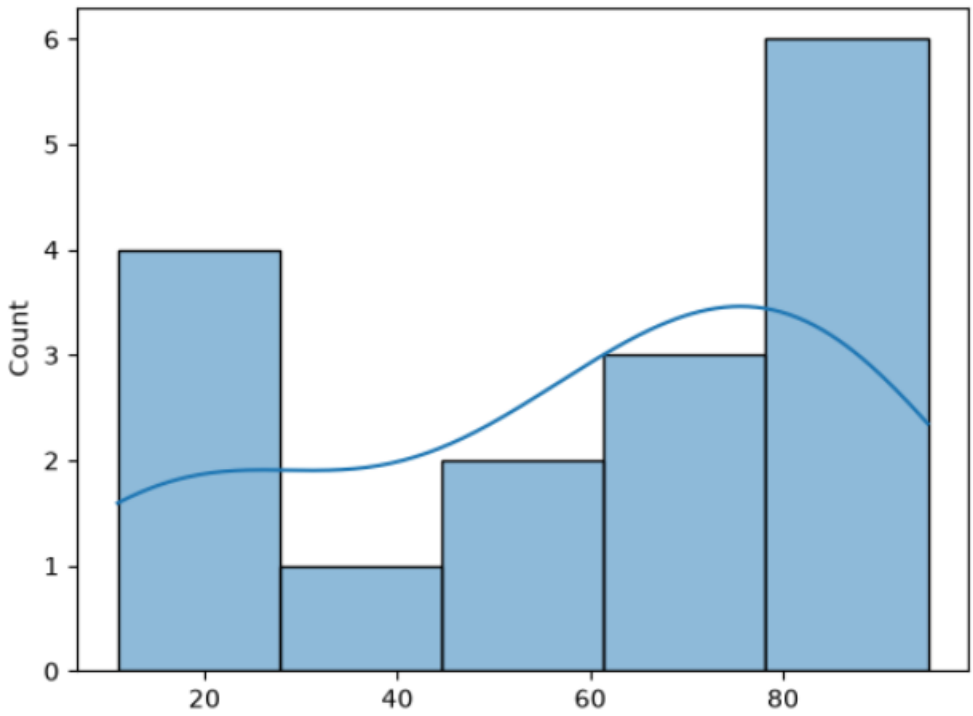
sns.histplot(new_array, kde=True)
plt.show()
```

```
[63 42 19 84 20 51 95 58 67 87 70 80 81 87 11 19]
```




```
lr1, ur1 = outDetection(new_array)
print(lr1, ur1)
final_array = new_array[(new_array > lr1) & (new_array < ur1)]
print(final_array)
sns.histplot(final_array, kde=True)
plt.show()
```

```
-31.375 149.625
[63 42 19 84 20 51 95 58 67 87 70 80 81 87 11 19]
```



In [2]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\Public\Downloads\feature scaling dataset.csv")
df
```

Out[2]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

	France	44.0	72000	No
	Spain	27.0	48000	Yes
	Germany	30.0	54000	No
	Spain	38.0	61000	No
	Germany	40.0	Nan	Yes
	France	35.0	58000	Yes
	Spain	NaN	52000	No
	France	48.0	79000	Yes
	NaN	50.0	83000	No
	France	37.0	67000	Yes

In [3]:
```
df.head()
```

Out[3]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

	France	44.0	72000	No
	Spain	27.0	48000	Yes
	Germany	30.0	54000	No
	Spain	38.0	61000	No
	Germany	40.0	Nan	Yes

In [ ]:
```
```

In [8]:
```
label=df.iloc[:,-1].values
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
```

```
age.fit(features[:,[1]])
SimpleImputer()

Salary.fit(features[:,[2]])

SimpleImputer()

SimpleImputer()

SimpleImputer()

features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
features
```

Out[8]:
```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       [nan, 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

In [9]:
```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```

Out[9]:
```
array([[1., 0., 0., 0.],
       [0., 0., 1., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 1., 0., 0.],
       [1., 0., 0., 0.],
       [0., 0., 1., 0.],
       [1., 0., 0., 0.],
       [0., 0., 0., 1.],
       [1., 0., 0., 0.]])
```

In [10]:
```
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
final_set
```

Out[10]:
```
array([[1.0, 0.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 0.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 0.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 0.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 0.0, 48.0, 79000.0],
       [0.0, 0.0, 0.0, 1.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

In [11]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
```

```
feat_standard_scaler=sc.transform(final_set)
feat_standard_scaler
```

Out[11]:
```
array([[ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01,  7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -5.00000000e-01,  1.52752523e+00,
        -3.33333333e-01, -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  2.00000000e+00, -6.54653671e-01,
        -3.33333333e-01, -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -5.00000000e-01,  1.52752523e+00,
        -3.33333333e-01, -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  2.00000000e+00, -6.54653671e-01,
        -3.33333333e-01,  1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01, -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -5.00000000e-01,  1.52752523e+00,
        -3.33333333e-01,  0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01,  1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01, -5.00000000e-01, -6.54653671e-01,
         3.00000000e+00,  1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -5.00000000e-01, -6.54653671e-01,
        -3.33333333e-01, -2.58340208e-01,  2.93712492e-01]])
```

In [12]:
```
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
feat_minmax_scaler
```

Out[12]:
```
array([[1.        , 0.        , 0.        , 0.        , 0.73913043,
        0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ,
        0.        ],
       [0.        , 1.        , 0.        , 0.        , 0.13043478,
        0.17142857],
       [0.        , 0.        , 1.        , 0.        , 0.47826087,
        0.37142857],
       [0.        , 1.        , 0.        , 0.        , 0.56521739,
        0.45079365],
       [1.        , 0.        , 0.        , 0.        , 0.34782609,
        0.28571429],
       [0.        , 0.        , 1.        , 0.        , 0.51207729,
        0.11428571],
       [1.        , 0.        , 0.        , 0.        , 0.91304348,
        0.88571429],
       [0.        , 0.        , 0.        , 1.        , 1.        ,
        1.        ],
       [1.        , 0.        , 0.        , 0.        , 0.43478261,
        0.54285714]])
```

In [ ]:
```

```

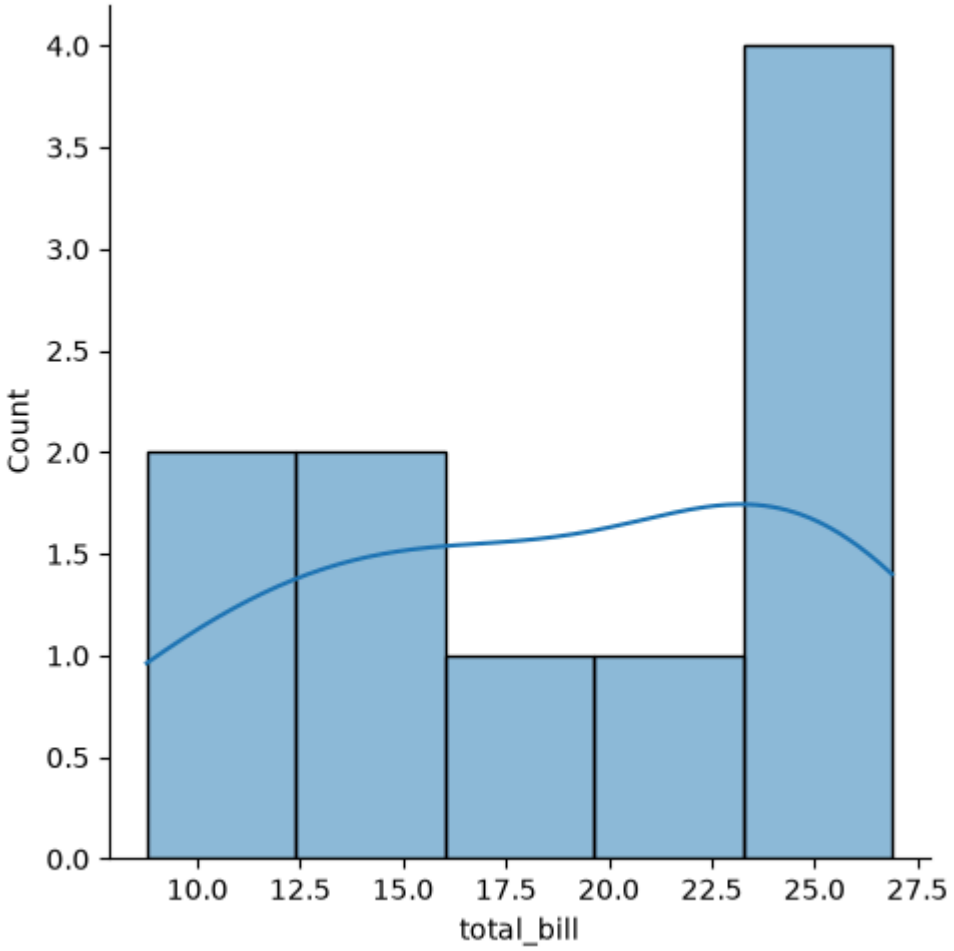
In [1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Create a sample dataset with the correct column names
tips = pd.DataFrame({
    'total_bill': [16.99, 10.34, 21.01, 23.68, 24.59,
                   25.29, 8.77, 26.88, 15.04, 14.78],
    'tip': [1.01, 1.66, 3.50, 3.31, 3.61,
            4.71, 2.00, 3.12, 1.96, 3.23],
    'sex': ['Female', 'Male', 'Male', 'Male', 'Female',
            'Male', 'Male', 'Male', 'Male', 'Female'],
    'smoker': ['No', 'No', 'No', 'No', 'No',
               'Yes', 'No', 'Yes', 'No', 'No'],
    'day': ['Sun'] * 10,
    'time': ['Dinner'] * 10,
    'size': [2, 3, 3, 2, 4, 4, 2, 4, 2, 2]
})

# Check the columns
print(tips.columns.tolist())

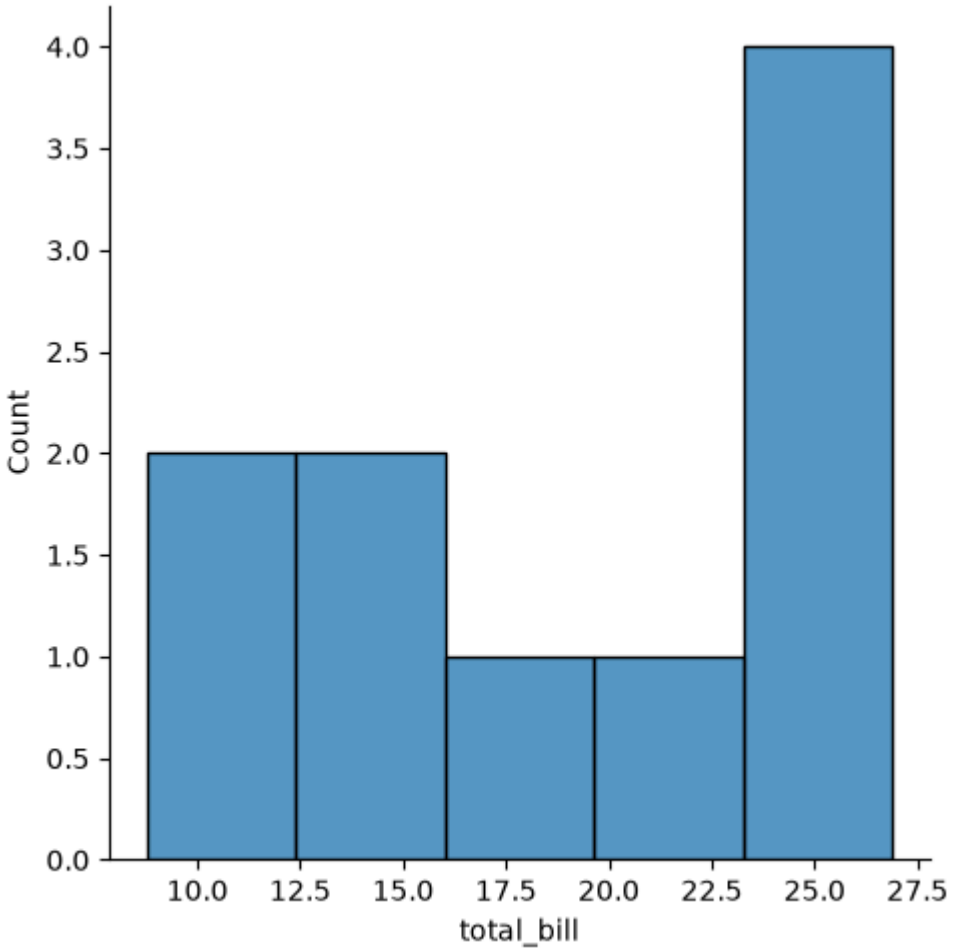
# Draw the distribution plot
sns.displot(data=tips, x='total_bill', kde=True)
plt.show()
```

```
['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size']
```



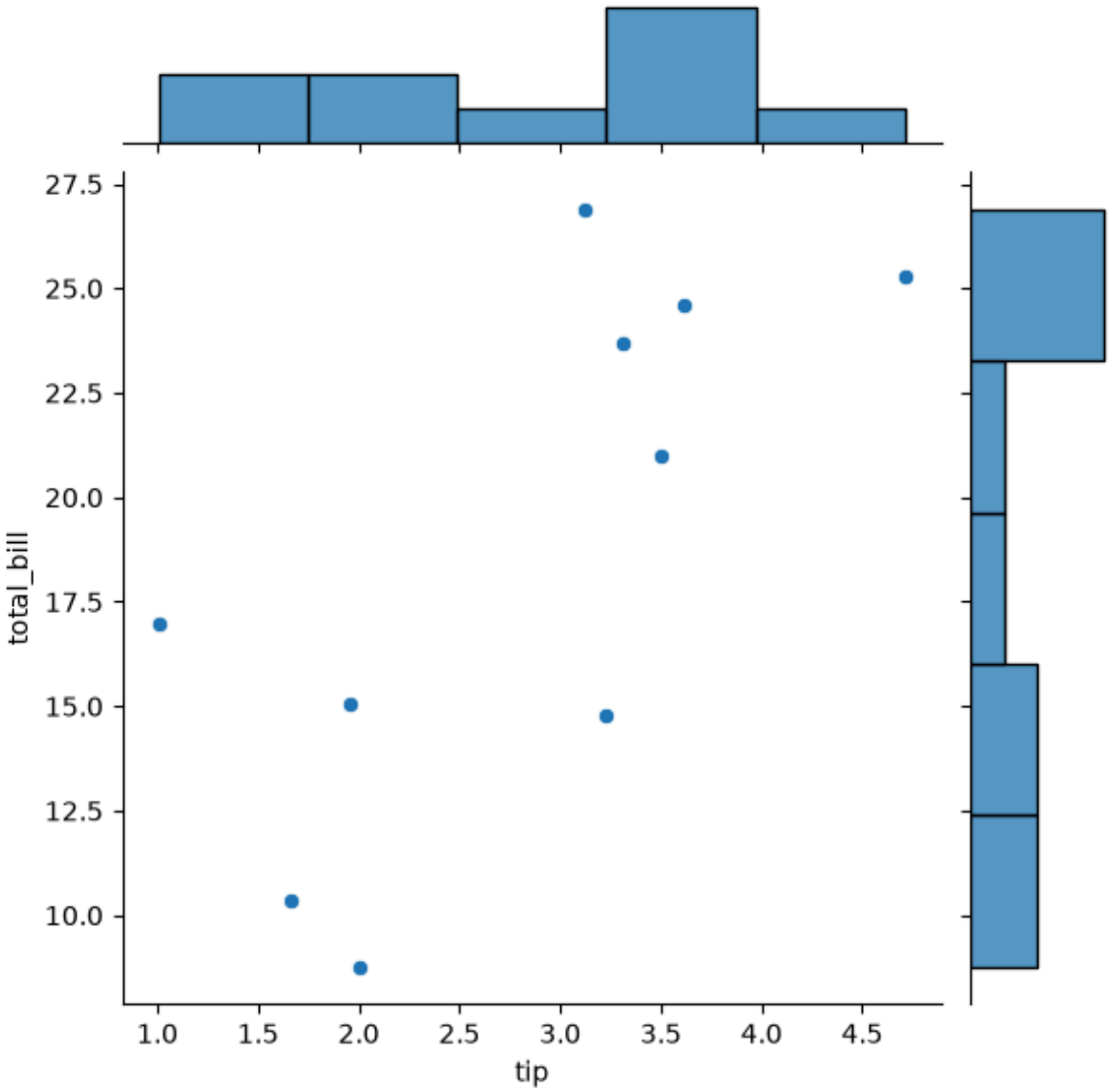
In [2]:
```
sns.displot(tips.total_bill,kde=False)
```

Out[2]: <seaborn.axisgrid.FacetGrid at 0x159e369ead0>



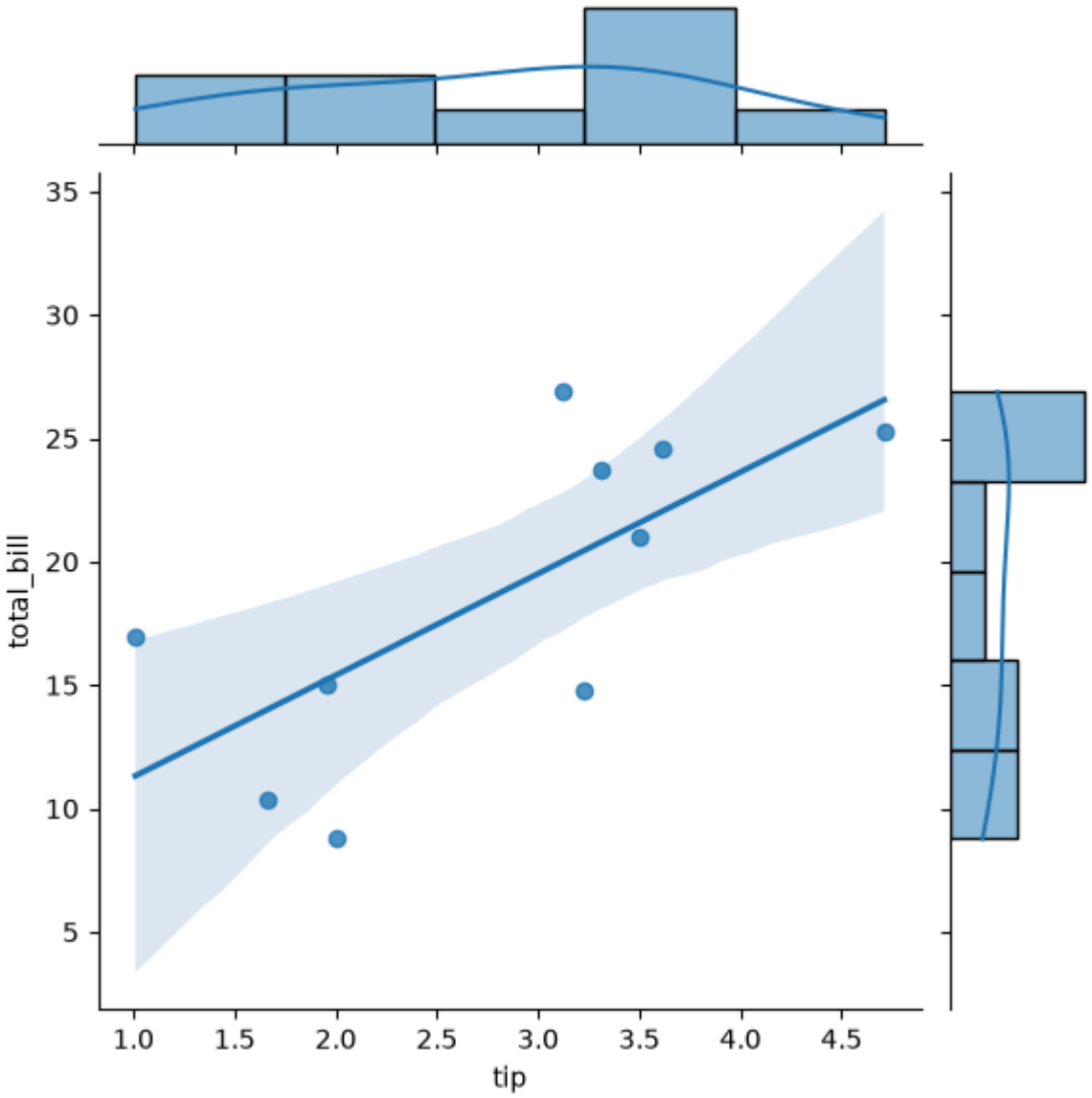
In [3]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill)
```

Out[3]: <seaborn.axisgrid.JointGrid at 0x159e37a1d30>



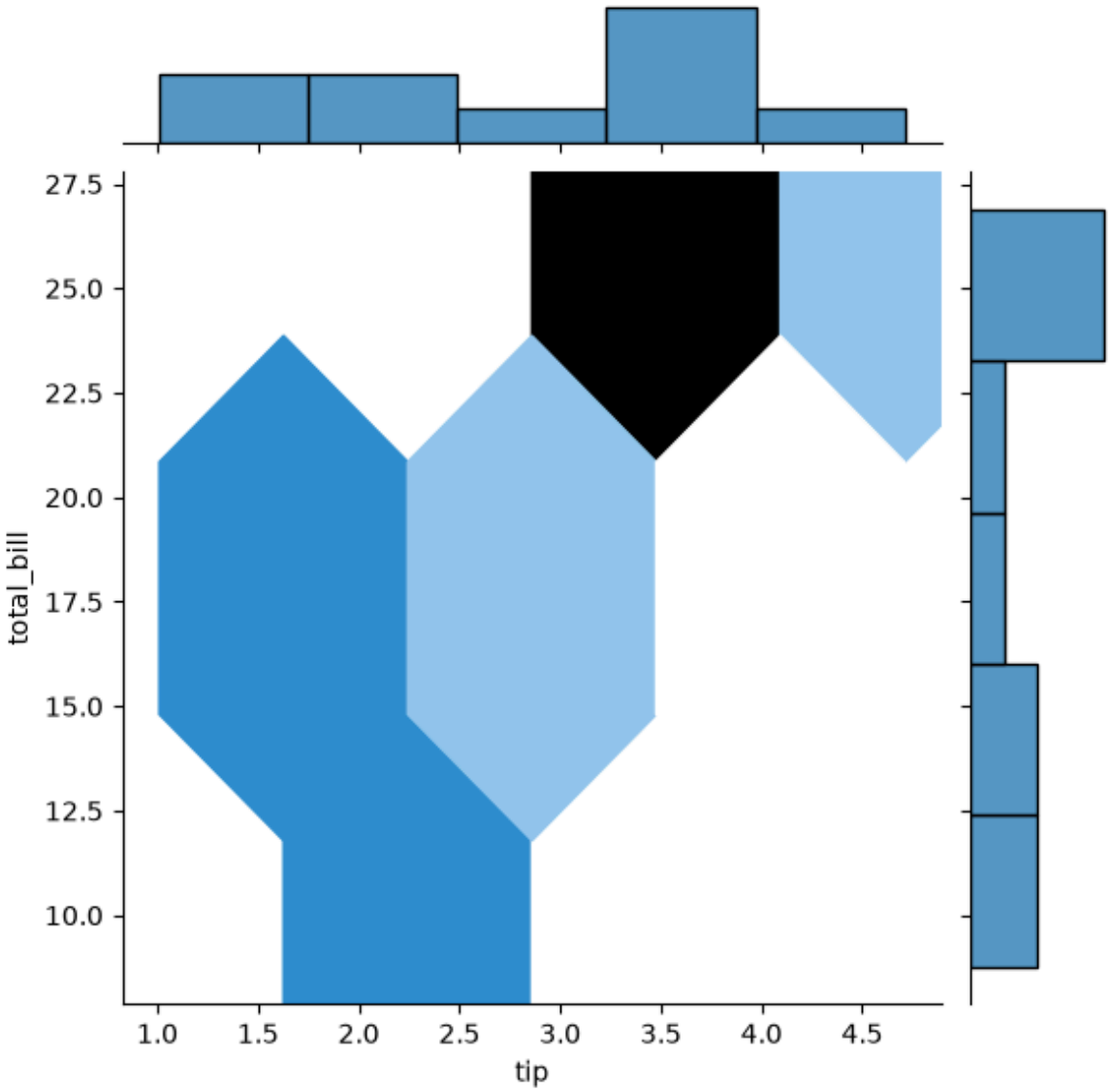
In [4]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
```

Out[4]: <seaborn.axisgrid.JointGrid at 0x159e49d5090>



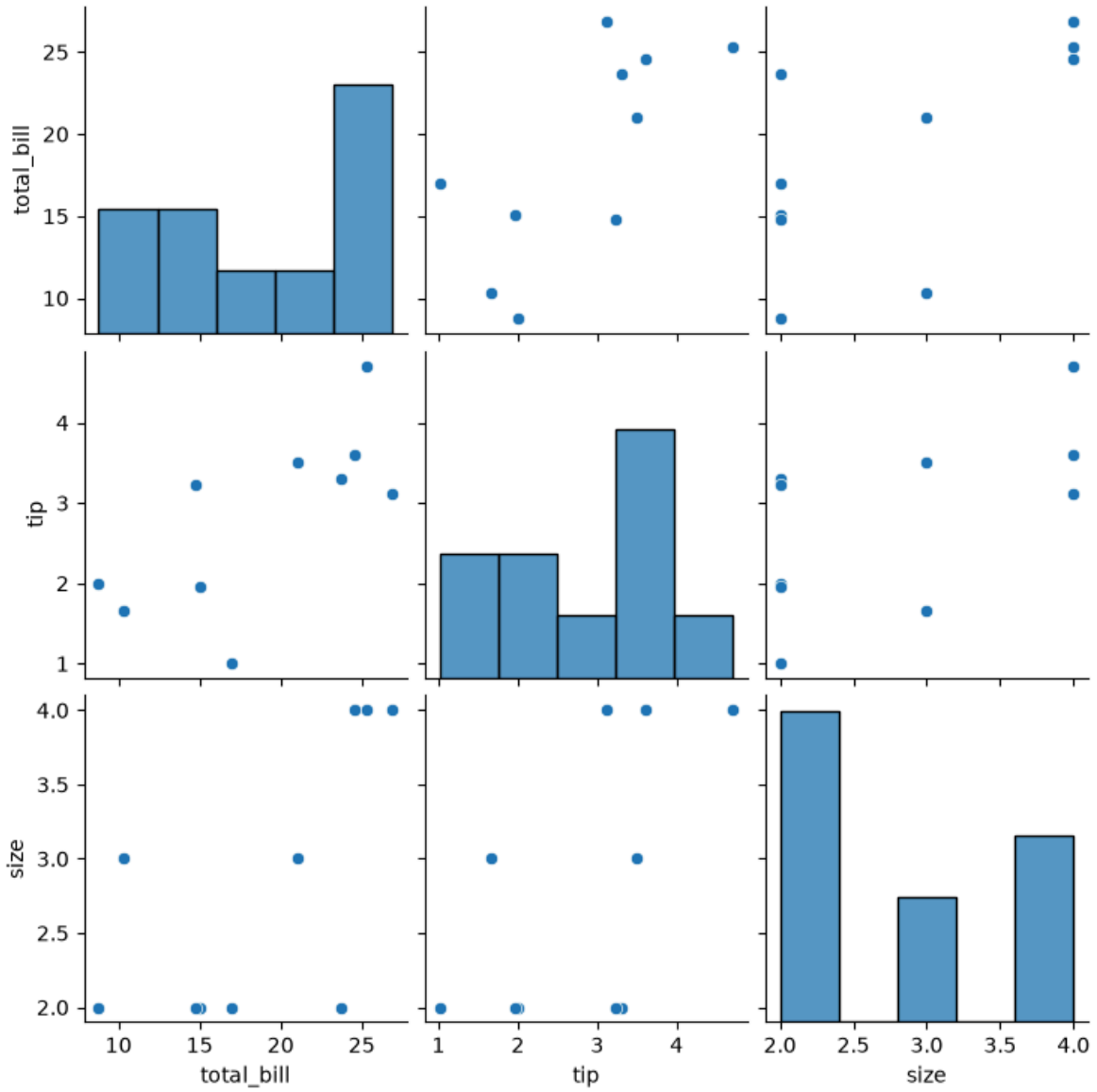
In [5]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
```

Out[5]: <seaborn.axisgrid.JointGrid at 0x159e49d4f50>



In [6]:
```
sns.pairplot(tips)
```

Out[6]: <seaborn.axisgrid.PairGrid at 0x159e53b5550>

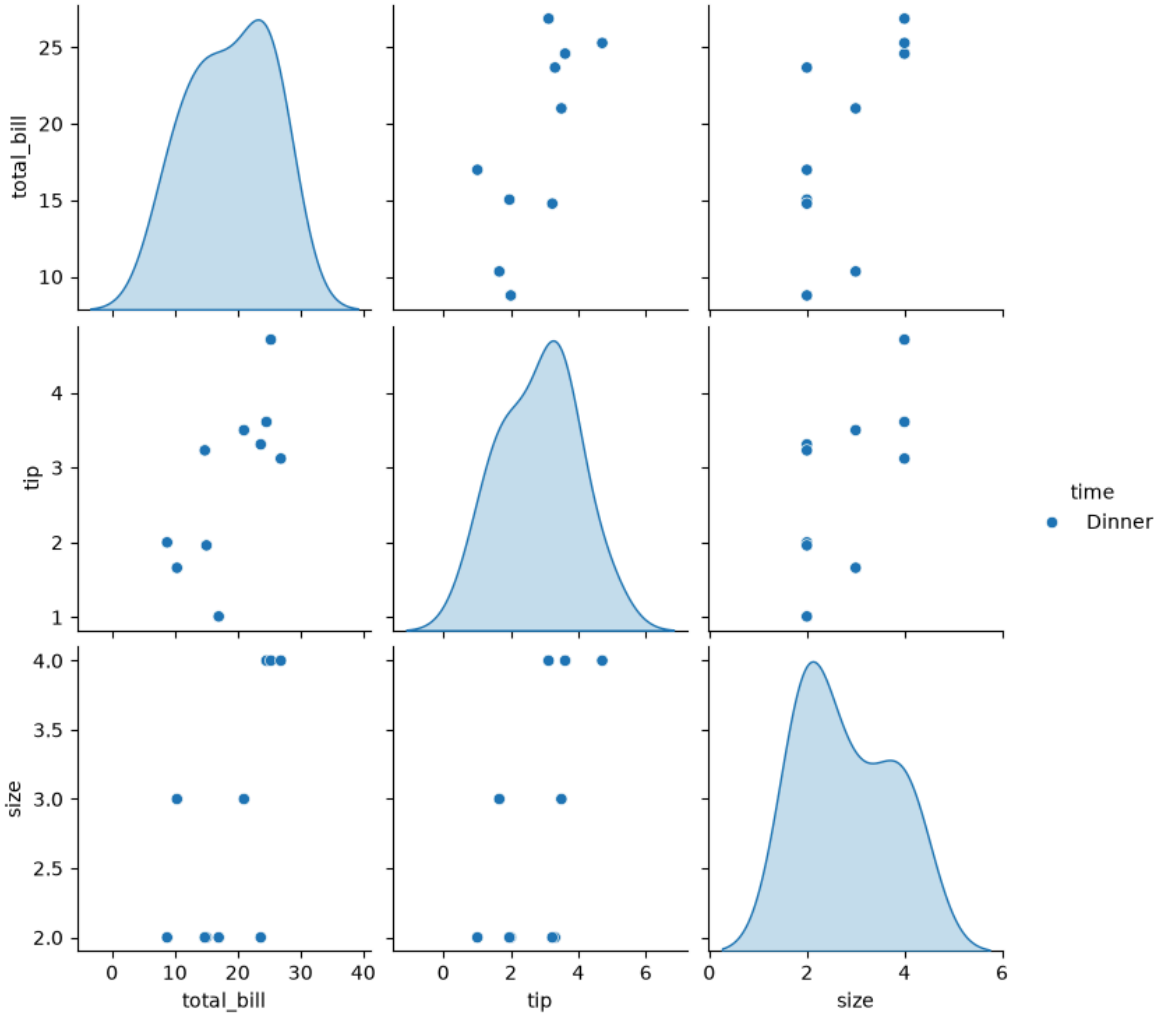


In [7]:
```
tips.time.value_counts()
```

Out[7]:
```
time
Dinner    10
Name: count, dtype: int64
```

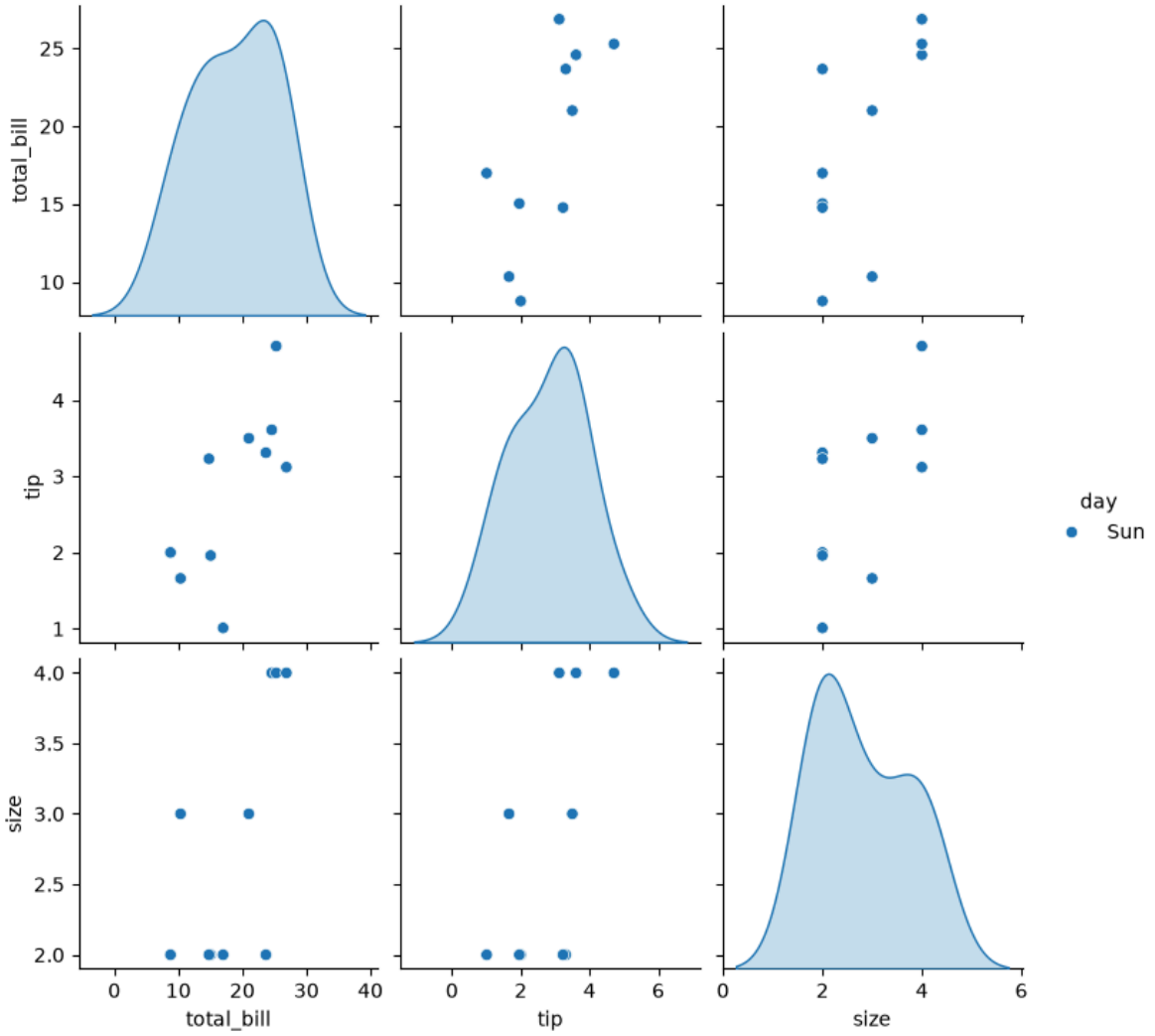
In [8]:
```
sns.pairplot(tips,hue='time')
```

Out[8]:
```
<seaborn.axisgrid.PairGrid at 0x159e5b1d090>
```



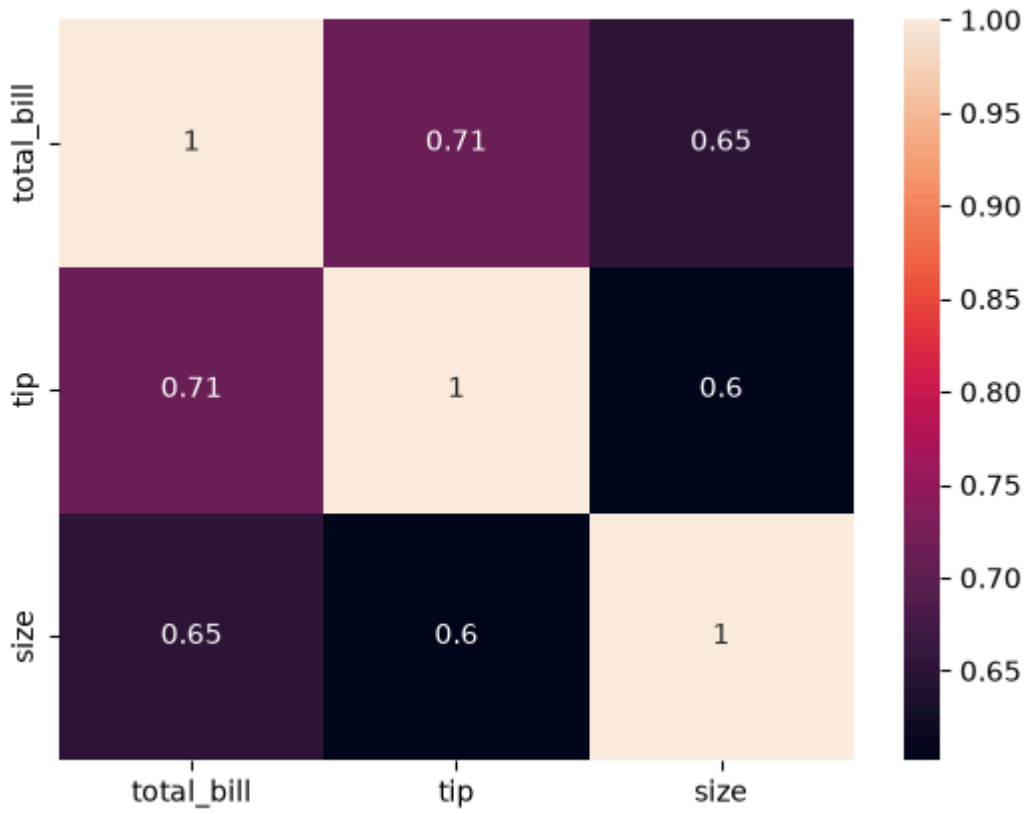
In [9]:
```
sns.pairplot(tips,hue='day')
```

Out[9]:
```
<seaborn.axisgrid.PairGrid at 0x159e63ba210>
```



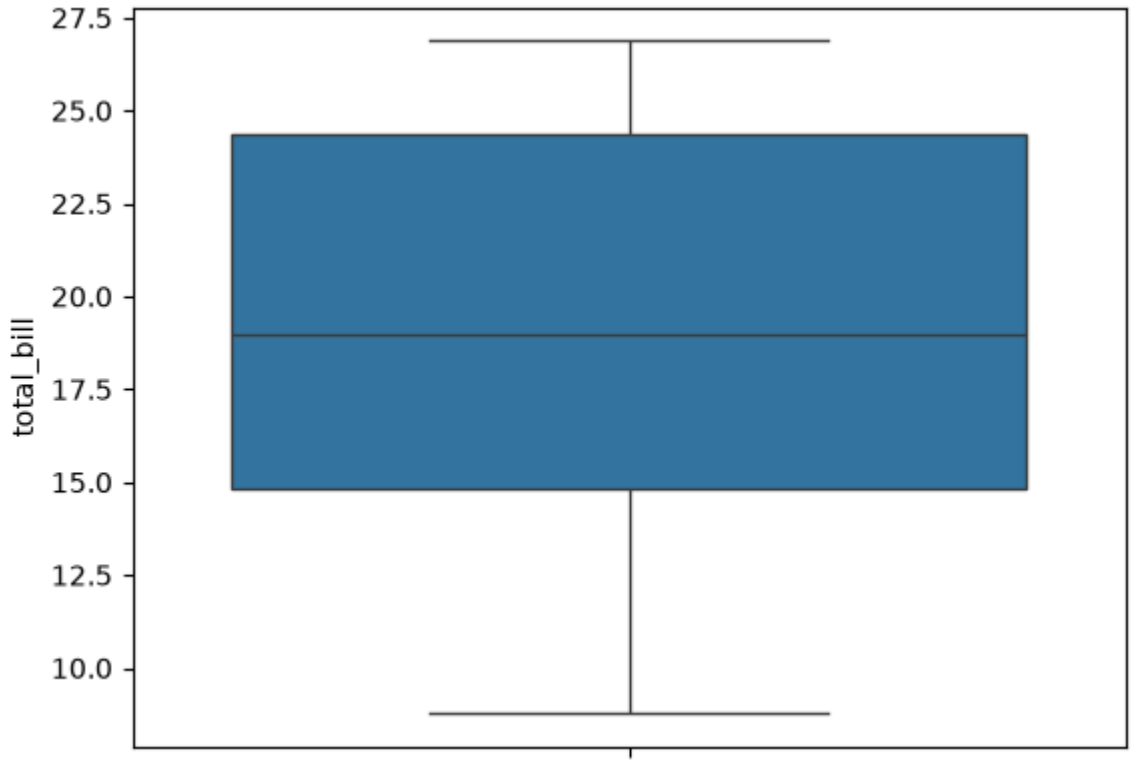
In [10]:
```
sns.heatmap(tips.corr(numeric_only=True),annot=True)
```

Out[10]: <Axes: >



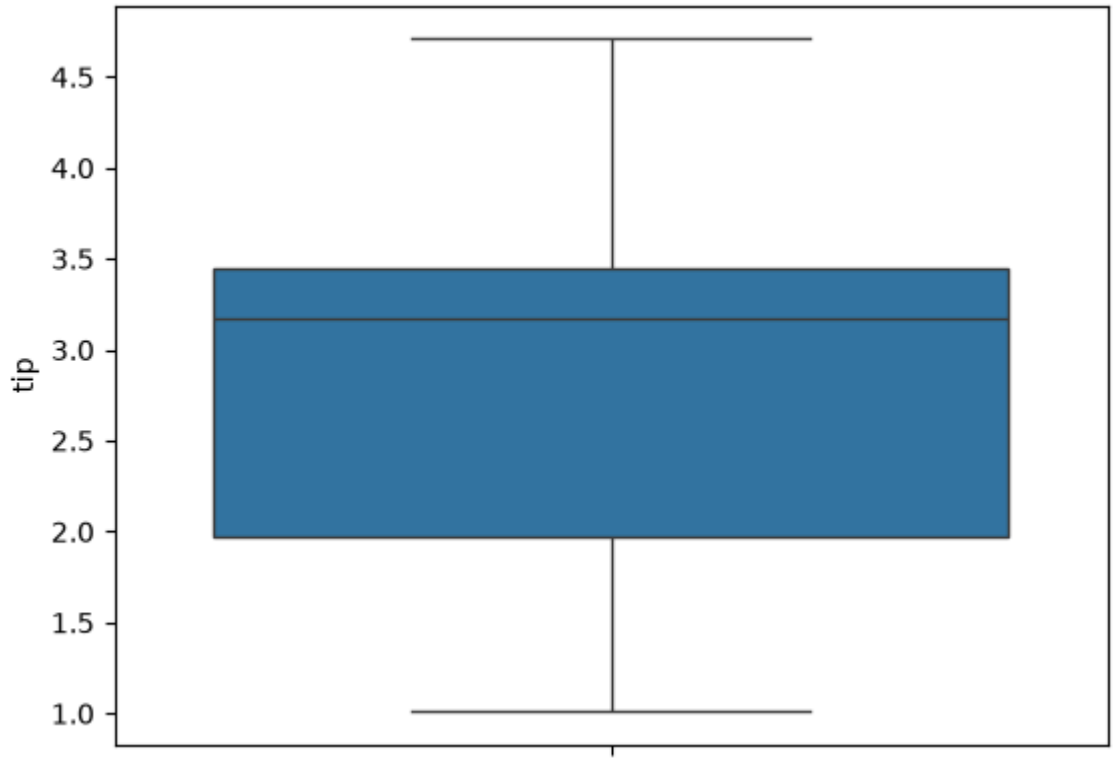
In [11]:
```
sns.boxplot(tips.total_bill)
```

Out[11]: <Axes: ylabel='total_bill'>



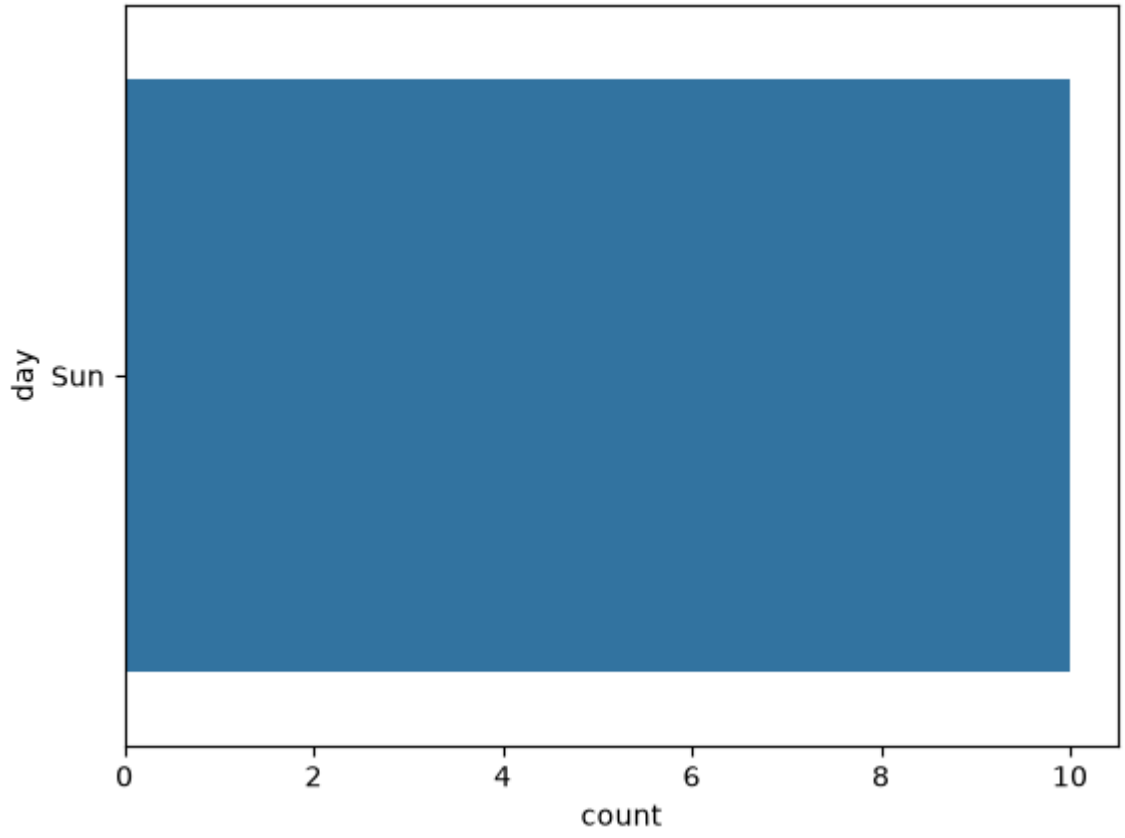
In [12]:
```
sns.boxplot(tips.tip)
```

Out[12]: <Axes: ylabel='tip'>



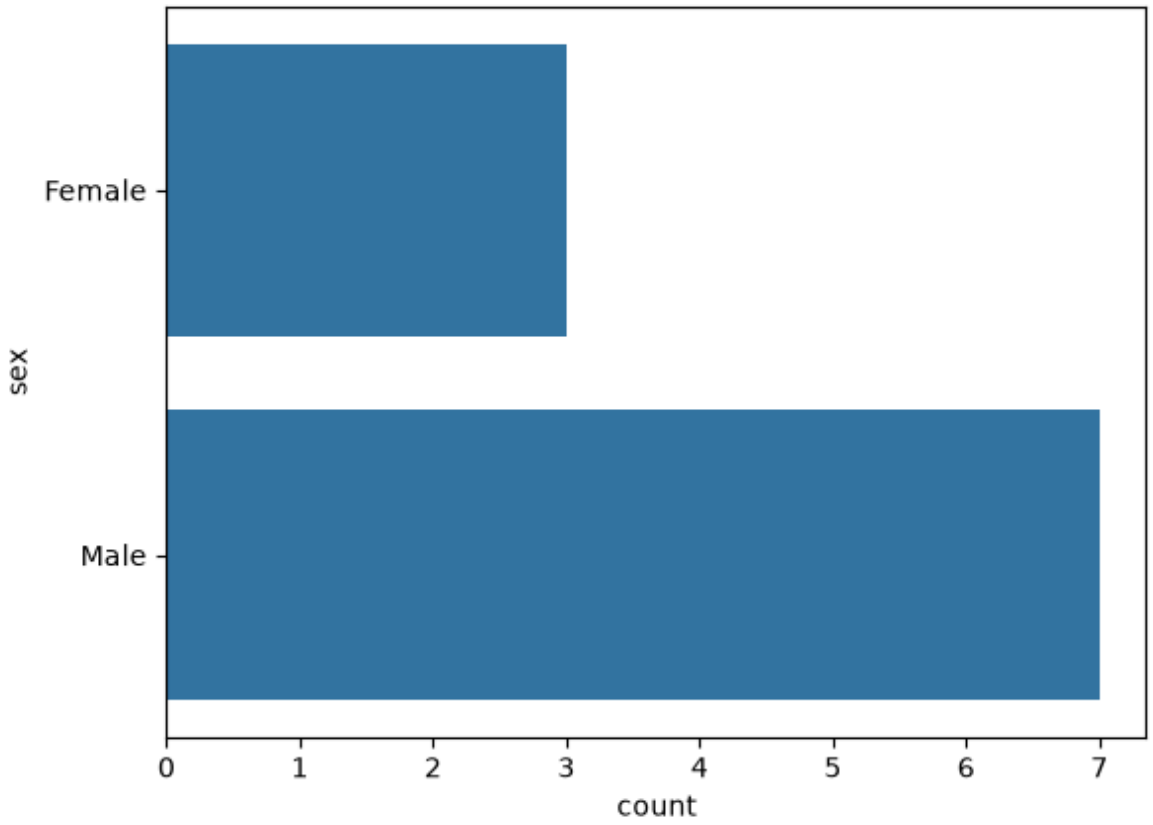
In [13]:
```
sns.countplot(tips.day)
```

Out[13]: <Axes: xlabel='count', ylabel='day'>



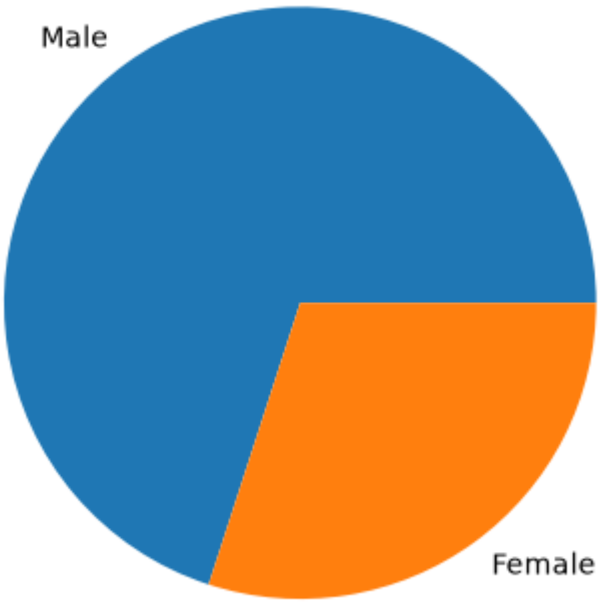
In [14]:
```
sns.countplot(tips.sex)
```

Out[14]: <Axes: xlabel='count', ylabel='sex'>



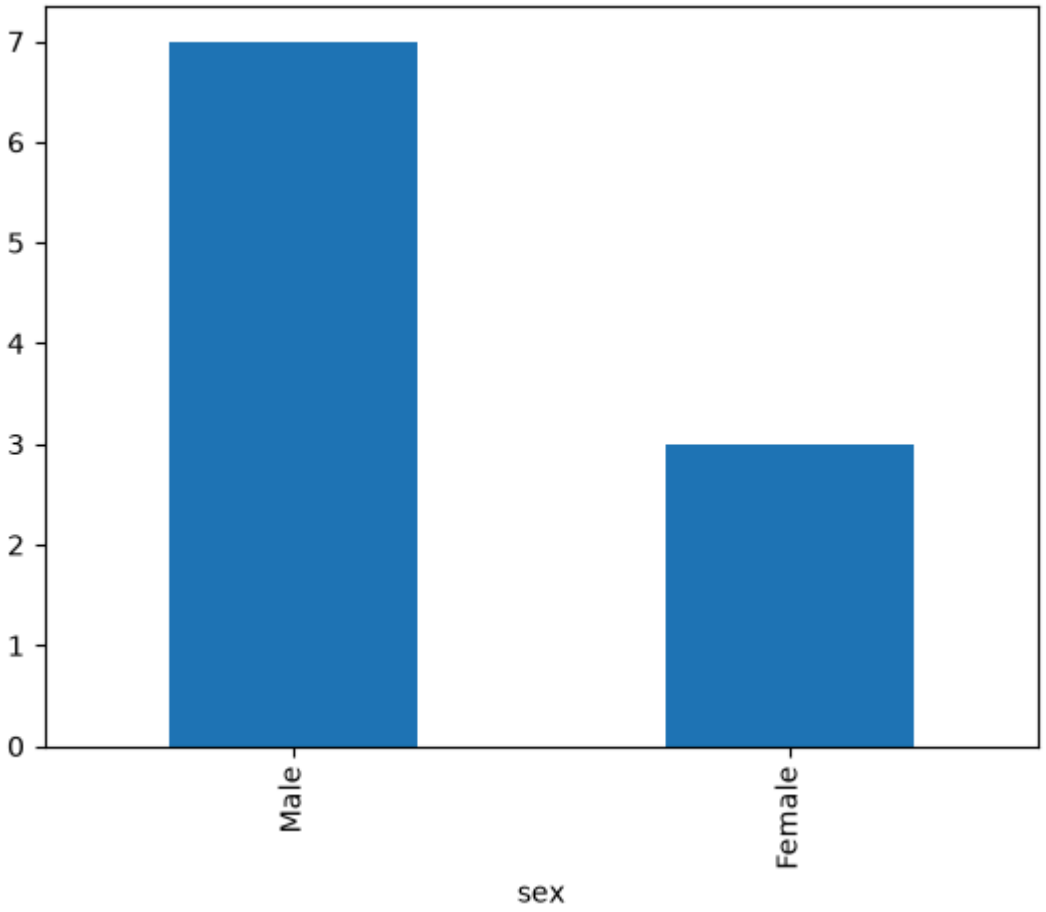
In [15]:
```
tips.sex.value_counts().plot(kind='pie')
```

Out[15]: <Axes: >



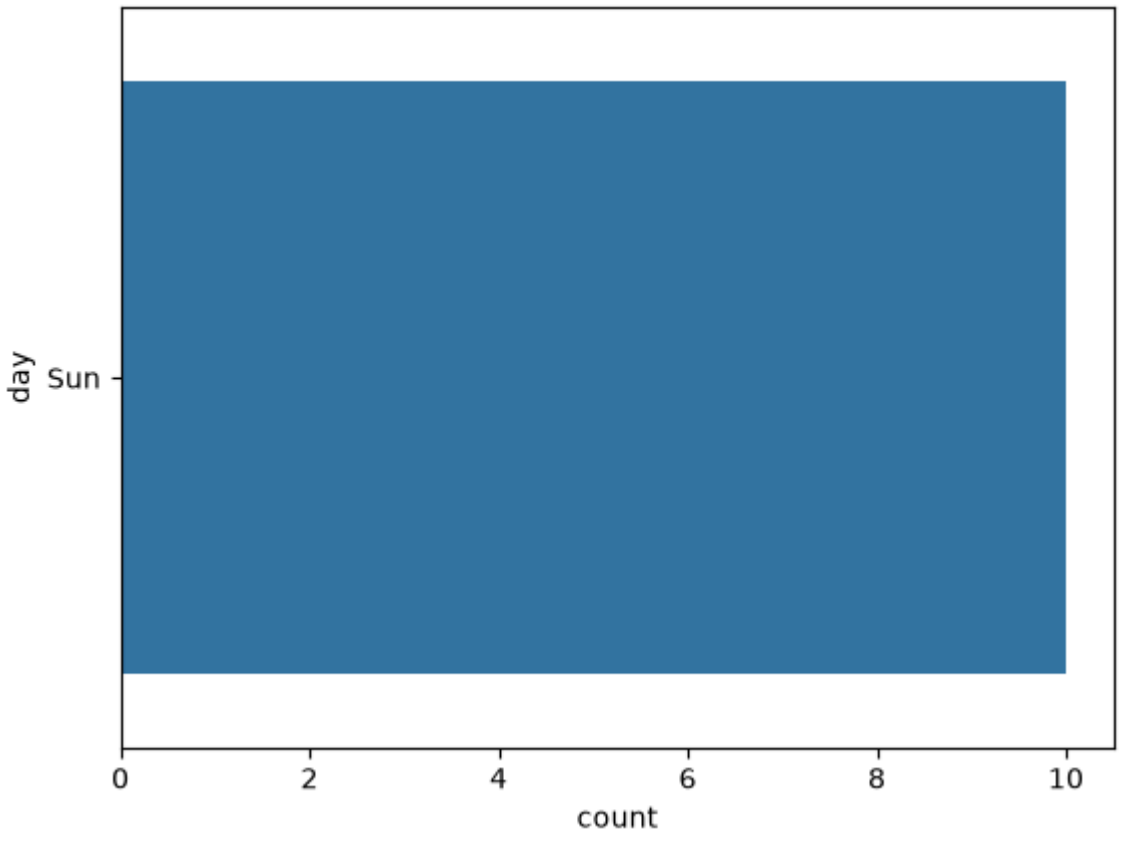
In [16]:
```
tips.sex.value_counts().plot(kind='bar')
```

Out[16]: <Axes: xlabel='sex'>



In [17]:
```
sns.countplot(tips[tips.time=='Dinner']['day'])
```

Out[17]: <Axes: xlabel='count', ylabel='day'>



In [ ]:

In [1]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\kyoga\Downloads\Salary_data - Salary_data.csv")
df
```

Out[1]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>YearsExperience</th> <th>Salary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> <th>21</th> <th>22</th> <th>23</th> <th>24</th> <th>25</th> <th>26</th> <th>27</th> <th>28</th> <th>29</th> </tbody> <tbody> </tbody>

1.1	39343
1.3	46205
1.5	37731
2.0	43525
2.2	39891
2.9	56642
3.0	60150
3.2	54445
3.2	64445
3.7	57189
3.9	63218
4.0	55794
4.0	56957
4.1	57081
4.5	61111
4.9	67938
5.1	66029
5.3	83088
5.9	81363
6.0	93940
6.8	91738
7.1	98273
7.9	101302
8.2	113812
8.7	109431
9.0	105582

9.5	116969
9.6	112635
10.3	122391
10.5	121872

In [2]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [3]:
```
df.dropna(inplace=True)
```

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

In [5]:
```
df.describe()
```

Out[5]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>YearsExperience</th> <th>Salary</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

30.000000	30.000000
5.313333	76003.000000
2.837888	27414.429785
1.100000	37731.000000
3.200000	56720.750000
4.700000	65237.000000
7.700000	100544.750000
10.500000	122391.000000

In [6]:
```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

In [11]:
```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)
```

In [12]:
```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

Out[12]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden] { display: none !important; }` so we also need the `!important` here to be able to override the default hidden behavior on the sphinx rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(--sklearn-color-text-on-default-background)); background-size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-items: center; background-color: var(--sklearn-color-background); padding-right: 1em; padding-left: 1em; } /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the default style - Estimators will overwrite some part of the style using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable { /* Default theme specific background. It is overwritten whether we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-

toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-

link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) { background-color: #f6f6f6; } .estimator-table

.parameters-table tr:hover td { background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104, 0.232); } /* `table td`is set in notebook with right text-align. We need to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell containing the link clickable */ a.param-doc-link::before { position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon { background-image: url(background-repeat: no-repeat; background-size: 14px 14px; background-position: 0; display: inline-block; width: 14px; height: 14px; cursor: pointer; border: none; padding: 0; background-color: transparent; } .copy-paste-icon:focus-visible { outline: 1px solid currentColor; outline-offset: 1px; } .features { font-family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color: var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap: 0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; } .features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before { content: "▸"; color: grey; } .features details[open] > summary .arrow::before { content: "▾"; } .features details:hover > summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features .features-container { max-height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-

color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover { background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style><body>
LinearRegression()
**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-1" type="checkbox" checked><label for="sk-estimator-id-1" class="sk-toggleable__label fitted sk-toggleable__label-arrow">
LinearRegression
?Documentation for LinearRegressioniFitted
</label>
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy fit_intercept to clipboard" onclick="copyToClipboard('fit_intercept', this.parentElement.nextElementSibling)" ></button>	fit_intercept fit_intercept: bool, default=Tru Whether to calculate the intercept for this mc set to False, no intercept will be u calculations (i.e. data is expected to be cent
<button type="button" class="copy-paste-icon" aria-label="Copy copy_X to clipboard" onclick="copyToClipboard('copy_X', this.parentElement.nextElementSibling)" ></button>	copy_X copy_X: bool, default=True If True be copied; else, it may be overw
<button type="button" class="copy-paste-icon" aria-label="Copy tol to clipboard" onclick="copyToClipboard('tol', this.parentElement.nextElementSibling)" ></button>	tol tol: float, default=1e-6 The precision solution (`coef_`) is determined b which specifies the convergence criterion underlying solver. `tol` is set as `atol` and `b :func:`scipy.sparse.linalg.lsqr` when fitting on s training data. `tol` is set as of :func:`scipy.linalg.lstsq` when fitting on training data.  .. versionadded:: 1.7 versionchanged:: 1.9  Now supported on dense interpreted as the `cond` para
<button type="button" class="copy-paste-icon" aria-label="Copy n_jobs to	n_jobs n_jobs: int, default=None The num jobs to use for the computation. This wi

clipboard" onclick="copyToClipboard('n_jobs', this.parentElement.nextElementSibling)" ></button>	provide speedup in case of sufficiently problems, that is if firstly `n_targets > secondly `X` is sparse or if `positive` is set to ``None`` means 1 un a :obj:`joblib.parallel_backend` context. ``-1`` r using all processors. See :term:`Glossary <n_ for more d
<button type="button" class="copy-paste-icon" aria-label="Copy positive to clipboard" onclick="copyToClipboard('positive', this.parentElement.nextElementSibling)" ></button>	positive positive: bool, default=False Wh to ``True``, forces the coefficients to be pc This option is only supported for dense array For a comparison between a linear regr model with positive constraints on the regr coefficients and a linear regression withou constraints,<b :ref:`sphx_glr_auto_examples_linear_model_plot_nr .. versionadded

</details>
<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th> <th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

coef_ coef_: array of shape (n_features,) or (n_targets, n_features) Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features.	ndarray[float64] (1, 1)	[[9423.82]]
intercept_ intercept_: float or array of shape (n_targets,) Independent term in the linear model. Set to 0.0 if `fit_intercept = False`.	ndarray[float64] (1,)	[25321.58]
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	1
rank_ rank_: int Rank of matrix `X`. Only available when `X` is dense.	int64	np.int64(1)
singular_ singular_: array of shape (min(X, y),) Singular values of `X`. Only available when `X` is dense.	ndarray[float64] (1,)	[14.11]

</details>

<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from the closest toggleable content const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `${paramPrefix}${text}` : text; const originalStyle = element.style; const computedStyle = window.getComputedStyle(element); const originalWidth = computedStyle.width; const originalHTML = element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width = originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!";

```
setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false; } document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent = element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected copy-paste icon is missing from the DOM structure'); return; } const paramName = element.parentElement.nextElementSibling .textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `${paramPrefix}${paramName}` : paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a `.features` block; its siblings * contain a `details` element and a table containing feature named. * @returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content = element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` "${row}"`); const formattedText = `[\n${rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ”', ''); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = 'âœ”'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_dat
* @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color');
```

const match = color.match(/^rgb\s*\(\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*\)\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [14]:
```
model.score(x_train,y_train)

model.score(x_test,y_test)

model.coef_

model.intercept_

import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)

print("Estimated Salary for {} years of experience is {}: " .format(yr_of
```

```
Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```

In [ ]:

In [1]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\kyoga\Downloads\Social_Network_Ads - Social_Net
df
```

Out[1]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>User ID</th> <th>Gender</th> <th>Age</th> <th>EstimatedSalary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>395</th> <th>396</th> <th>397</th> <th>398</th> <th>399</th> </tbody> <tbody></tbody>

15624510	Male	19	19000	0
15810944	Male	35	20000	0
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	0
...	...	...	...	...
15691863	Female	46	41000	1
15706071	Male	51	23000	1
15654296	Female	50	20000	1
15755018	Male	36	33000	0
15594041	Female	49	36000	1

400 rows × 5 columns

In [2]:
```
df.head()
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>User ID</th> <th>Gender</th> <th>Age</th> <th>EstimatedSalary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

15624510	Male	19	19000	0
15810944	Male	35	20000	0
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	0

In [3]:
```
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
```

Out[3]:
```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
       [    48,  29000],
       [    45,  22000],
       [    47,  49000],
       [    48,  41000],
       [    45,  22000],
       [    46,  23000],
       [    47,  20000],
       [    49,  28000],
       [    47,  30000],
       [    29,  43000],
       [    31,  18000],
       [    31,  74000],
       [    27, 137000],
       [    21,  16000],
       [    28,  44000],
       [    27,  90000],
       [    35,  27000],
       [    33,  28000],
       [    30,  49000],
       [    26,  72000],
       [    27,  31000],
       [    27,  17000],
       [    33,  51000],
       [    35, 108000],
       [    30,  15000],
       [    28,  84000],
       [    23,  20000],
       [    25,  79000],
       [    27,  54000],
       [    30, 135000],
       [    31,  89000],
       [    24,  32000],
       [    18,  44000],
       [    29,  83000],
       [    35,  23000],
       [    27,  58000],
       [    24,  55000],
       [    23,  48000],
       [    28,  79000],
       [    22,  18000],
       [    32, 117000],
```

```
[    27,  20000],
[    25,  87000],
[    23,  66000],
[    32, 120000],
[    59,  83000],
[    24,  58000],
[    24,  19000],
[    23,  82000],
[    22,  63000],
[    31,  68000],
[    25,  80000],
[    24,  27000],
[    20,  23000],
[    33, 113000],
[    32,  18000],
[    34, 112000],
[    18,  52000],
[    22,  27000],
[    28,  87000],
[    26,  17000],
[    30,  80000],
[    39,  42000],
[    20,  49000],
[    35,  88000],
[    30,  62000],
[    31, 118000],
[    24,  55000],
[    28,  85000],
[    26,  81000],
[    35,  50000],
[    22,  81000],
[    30, 116000],
[    26,  15000],
[    29,  28000],
[    29,  83000],
[    35,  44000],
[    35,  25000],
[    28, 123000],
[    35,  73000],
[    28,  37000],
[    27,  88000],
[    28,  59000],
[    32,  86000],
[    33, 149000],
[    19,  21000],
[    21,  72000],
[    26,  35000],
[    27,  89000],
[    26,  86000],
[    38,  80000],
[    39,  71000],
[    37,  71000],
[    38,  61000],
[    37,  55000],
[    42,  80000],
[    40,  57000],
[    35,  75000],
[    36,  52000],
[    40,  59000],
[    41,  59000],
```

```
[    36,  75000],
[    37,  72000],
[    40,  75000],
[    35,  53000],
[    41,  51000],
[    39,  61000],
[    42,  65000],
[    26,  32000],
[    30,  17000],
[    26,  84000],
[    31,  58000],
[    33,  31000],
[    30,  87000],
[    21,  68000],
[    28,  55000],
[    23,  63000],
[    20,  82000],
[    30, 107000],
[    28,  59000],
[    19,  25000],
[    19,  85000],
[    18,  68000],
[    35,  59000],
[    30,  89000],
[    34,  25000],
[    24,  89000],
[    27,  96000],
[    41,  30000],
[    29,  61000],
[    20,  74000],
[    26,  15000],
[    41,  45000],
[    31,  76000],
[    36,  50000],
[    40,  47000],
[    31,  15000],
[    46,  59000],
[    29,  75000],
[    26,  30000],
[    32, 135000],
[    32, 100000],
[    25,  90000],
[    37,  33000],
[    35,  38000],
[    33,  69000],
[    18,  86000],
[    22,  55000],
[    35,  71000],
[    29, 148000],
[    29,  47000],
[    21,  88000],
[    34, 115000],
[    26, 118000],
[    34,  43000],
[    34,  72000],
[    23,  28000],
[    35,  47000],
[    25,  22000],
[    24,  23000],
[    31,  34000],
```

```
[    26,  16000],
[    31,  71000],
[    32, 117000],
[    33,  43000],
[    33,  60000],
[    31,  66000],
[    20,  82000],
[    33,  41000],
[    35,  72000],
[    28,  32000],
[    24,  84000],
[    19,  26000],
[    29,  43000],
[    19,  70000],
[    28,  89000],
[    34,  43000],
[    30,  79000],
[    20,  36000],
[    26,  80000],
[    35,  22000],
[    35,  39000],
[    49,  74000],
[    39, 134000],
[    41,  71000],
[    58, 101000],
[    47,  47000],
[    55, 130000],
[    52, 114000],
[    40, 142000],
[    46,  22000],
[    48,  96000],
[    52, 150000],
[    59,  42000],
[    35,  58000],
[    47,  43000],
[    60, 108000],
[    49,  65000],
[    40,  78000],
[    46,  96000],
[    59, 143000],
[    41,  80000],
[    35,  91000],
[    37, 144000],
[    60, 102000],
[    35,  60000],
[    37,  53000],
[    36, 126000],
[    56, 133000],
[    40,  72000],
[    42,  80000],
[    35, 147000],
[    39,  42000],
[    40, 107000],
[    49,  86000],
[    38, 112000],
[    46,  79000],
[    40,  57000],
[    37,  80000],
[    46,  82000],
[    53, 143000],
```

```
[    42, 149000],
[    38,  59000],
[    50,  88000],
[    56, 104000],
[    41,  72000],
[    51, 146000],
[    35,  50000],
[    57, 122000],
[    41,  52000],
[    35,  97000],
[    44,  39000],
[    37,  52000],
[    48, 134000],
[    37, 146000],
[    50,  44000],
[    52,  90000],
[    41,  72000],
[    40,  57000],
[    58,  95000],
[    45, 131000],
[    35,  77000],
[    36, 144000],
[    55, 125000],
[    35,  72000],
[    48,  90000],
[    42, 108000],
[    40,  75000],
[    37,  74000],
[    47, 144000],
[    40,  61000],
[    43, 133000],
[    59,  76000],
[    60,  42000],
[    39, 106000],
[    57,  26000],
[    57,  74000],
[    38,  71000],
[    49,  88000],
[    52,  38000],
[    50,  36000],
[    59,  88000],
[    35,  61000],
[    37,  70000],
[    52,  21000],
[    48, 141000],
[    37,  93000],
[    37,  62000],
[    48, 138000],
[    41,  79000],
[    37,  78000],
[    39, 134000],
[    49,  89000],
[    55,  39000],
[    37,  77000],
[    35,  57000],
[    36,  63000],
[    42,  73000],
[    43, 112000],
[    45,  79000],
[    46, 117000],
```

```
[    58,  38000],
[    48,  74000],
[    37, 137000],
[    37,  79000],
[    40,  60000],
[    42,  54000],
[    51, 134000],
[    47, 113000],
[    36, 125000],
[    38,  50000],
[    42,  70000],
[    39,  96000],
[    38,  50000],
[    49, 141000],
[    39,  79000],
[    39,  75000],
[    54, 104000],
[    35,  55000],
[    45,  32000],
[    36,  60000],
[    52, 138000],
[    53,  82000],
[    41,  52000],
[    48,  30000],
[    48, 131000],
[    41,  60000],
[    41,  72000],
[    42,  75000],
[    36, 118000],
[    47, 107000],
[    38,  51000],
[    48, 119000],
[    42,  65000],
[    40,  65000],
[    57,  60000],
[    36,  54000],
[    58, 144000],
[    35,  79000],
[    38,  55000],
[    39, 122000],
[    53, 104000],
[    35,  75000],
[    38,  65000],
[    47,  51000],
[    47, 105000],
[    41,  63000],
[    53,  72000],
[    54, 108000],
[    39,  77000],
[    38,  61000],
[    38, 113000],
[    37,  75000],
[    42,  90000],
[    37,  57000],
[    36,  99000],
[    60,  34000],
[    54,  70000],
[    41,  72000],
[    40,  71000],
[    42,  54000],
```

```
[    43, 129000],
[    53,  34000],
[    47,  50000],
[    42,  79000],
[    42, 104000],
[    59,  29000],
[    58,  47000],
[    46,  88000],
[    38,  71000],
[    54,  26000],
[    60,  46000],
[    60,  83000],
[    39,  73000],
[    59, 130000],
[    37,  80000],
[    46,  32000],
[    46,  74000],
[    42,  53000],
[    41,  87000],
[    58,  23000],
[    42,  64000],
[    48,  33000],
[    44, 139000],
[    49,  28000],
[    57,  33000],
[    56,  60000],
[    49,  39000],
[    39,  71000],
[    47,  34000],
[    48,  35000],
[    48,  33000],
[    47,  23000],
[    45,  45000],
[    60,  42000],
[    39,  59000],
[    46,  41000],
[    51,  23000],
[    50,  20000],
[    36,  33000],
[    49,  36000]])
```

In [4]:
```
label
```

Out[4]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1])
```

In [10]:
```
%pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in .\AppData\Local\Python\pyth
oncore-3.14-64\Lib\site-packages (1.9.1)
Requirement already satisfied: numpy>=1.24.1 in .\AppData\Local\Python\pyt
honcore-3.14-64\Lib\site-packages (from scikit-learn) (2.5.2)
Requirement already satisfied: scipy>=1.10.0 in .\AppData\Local\Python\pyt
honcore-3.14-64\Lib\site-packages (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib>=1.4.0 in .\AppData\Local\Python\pyt
honcore-3.14-64\Lib\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in .\AppData\Local\Python\p
ythoncore-3.14-64\Lib\site-packages (from scikit-learn) (2.27.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in .\AppData\Local\Pyt
hon\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in .\AppData\Local\Python
\pythoncore-3.14-64\Lib\site-packages (from joblib>=1.4.0->scikit-learn)
(3.1.2)
Note: you may need to restart the kernel to use updated packages.
```

In [13]:
```
%whos
```

```
Variable                Type        Data/Info
------------------------------------------
LogisticRegression      type        <class 'sklearn.linear_mo<...>stic.Log
isticRegression'>
classification_report   function    <function classification_<...>rt at 0x
0000022FC79BEF00>
i                       int         1
train_test_split        function    <function train_test_split at 0x000002
2FC7A27C10>
```

In [6]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

print("Import successful")
```

```
Import successful
```

In [12]:
```
import pandas as pd

df = pd.read_csv(r"C:\Users\kyoga\Downloads\logistic_regression_features_
```

```
print(df.head())
print(df.columns)
```

```
   Hours_Studied  Attendance_Percent  Previous_Score  Pass_Label
0              2                  55              35           0
1              3                  60              40           0
2              1                  50              30           0
3              4                  65              45           0
4              2                  70              38           0
Index(['Hours_Studied', 'Attendance_Percent', 'Previous_Score', 'Pass_Labe
l'], dtype='str')
```

In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
```

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv(r"C:\Users\Public\Downloads\Mall_Customers.csv")
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    str
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), str(1)
memory usage: 7.9 KB
```

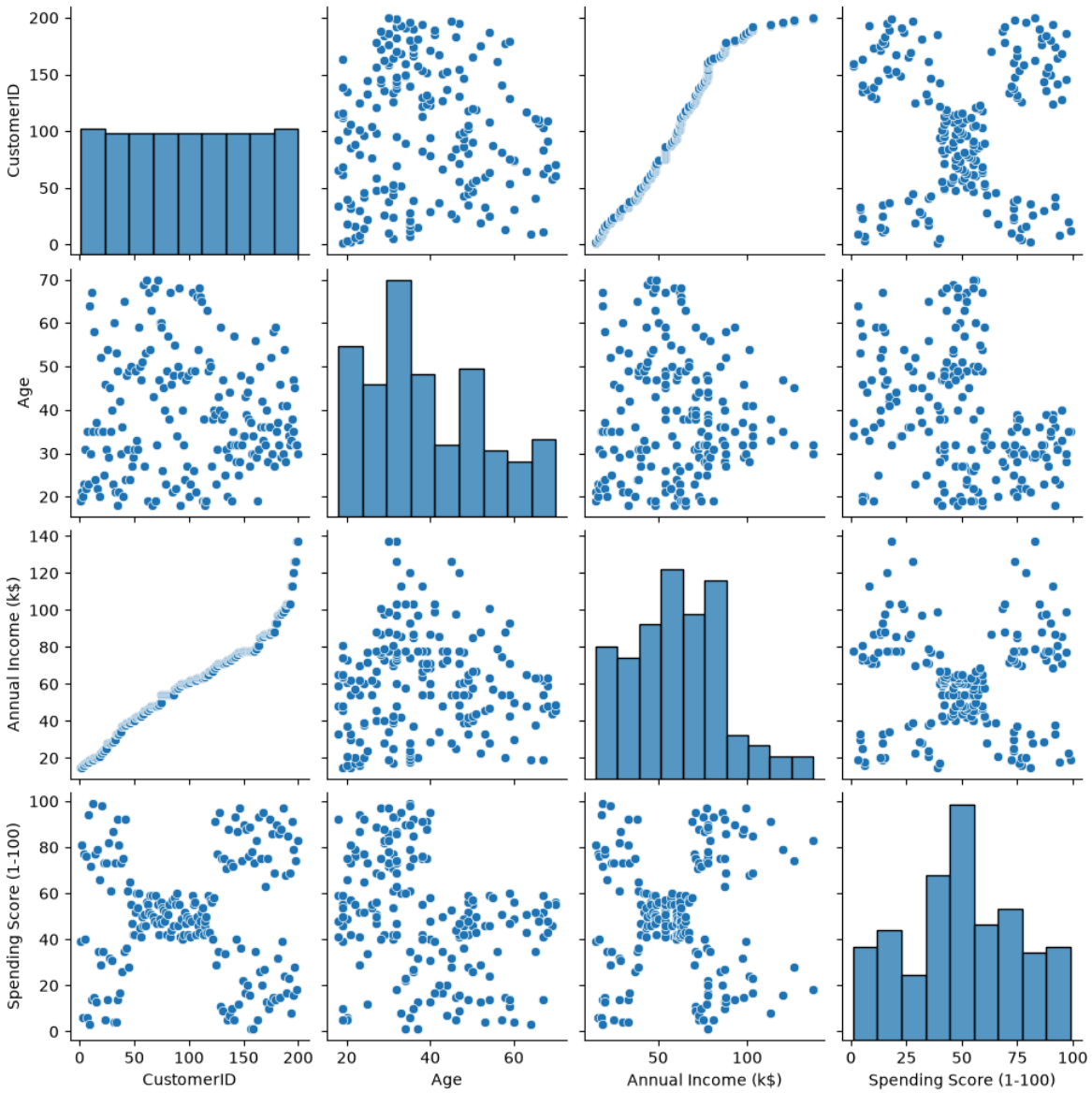
In [2]:
```
df.head()
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Gender</th> <th>Age</th> <th>Annual Income (k$)</th> <th>Spending Score (1-100)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

1		Male	19	15	39
2		Male	21	15	81
3	Female	20	16	6	
4	Female	23	16	77	
5	Female	31	17	40	

In [3]:
```
sns.pairplot(df)
```

Out[3]: <seaborn.axisgrid.PairGrid at 0x27c05a4b0e0>



In [3]:
```
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
```

Out[3]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden] { display: none !important; }` so we also need the `!important` here to be able to override the default hidden behavior on the sphinx rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(--sklearn-color-text-on-default-background)); background-size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-items: center; background-color: var(--sklearn-color-background); padding-right: 1em; padding-left: 1em; } /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the default style - Estimators will overwrite some part of the style using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable { /* Default theme specific background. It is overwritten whether we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-

toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-

link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) { background-color: #f6f6f6; } .estimator-table

.parameters-table tr:hover td { background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104, 0.232); } /* `table td`is set in notebook with right text-align. We need to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell containing the link clickable */ a.param-doc-link::before { position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon { background-image: url(background-repeat: no-repeat; background-size: 14px 14px; background-position: 0; display: inline-block; width: 14px; height: 14px; cursor: pointer; border: none; padding: 0; background-color: transparent; } .copy-paste-icon:focus-visible { outline: 1px solid currentColor; outline-offset: 1px; } .features { font-family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color: var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap: 0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; } .features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before { content: "▸"; color: grey; } .features details[open] > summary .arrow::before { content: "▾"; } .features details:hover > summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features .features-container { max-height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-

color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover { background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style><body>
KMeans(n_clusters=5)
**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-1" type="checkbox" checked><label for="sk-estimator-id-1" class="sk-toggleable__label sk-toggleable__label-arrow">
KMeans
?Documentation for KMeansiNot fitted
</label>
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy n_clusters to clipboard" onclick="copyToClipboard('n_clusters', this.parentElement.nextElementSibling)" ></button>	n_clusters n_clusters: int, default=8 Th number of centroids to generate. Fo to :ref:`sphx_glr_auto_examples_cl
<button type="button" class="copy-paste-icon" aria-label="Copy init to clipboard" onclick="copyToClipboard('init', this.parentElement.nextElementSibling)" ></button>	init init: {'k-means++', 'random'}, callable or default='k-means++' Method for initializa cluster centroids using sampling based on an emp contribution to the overall inertia. This technic implemented is "greedy k-means++". It differs fro trials at each sampling step and choosing 'random': choose `n_clusters` observation centroids. * If an array is passed, it shou gives the initial centers. * If a call n_clusters and a random state and return an initia see :ref:`sphx_glr_auto_examples_clus evalu example :ref:`sphx_glr_auto_examples_cluster
<button type="button" class="copy-paste-icon" aria-label="Copy n_init to clipboard" onclick="copyToClipboard('n_init',	n_init n_init: 'auto' or int, default='auto' run with different centroid seeds. T consecutive runs in terms of inertia. Several ru dimensional problems (see :ref:`kmeans_spars the number of runs depends on the value of ini

this.parentElement.nextElementSibling)" ></button>	callable; 1 if using `init='k-means++'` or `ini 1.2  Added 'auto' option for `n_init`. ..
<button type="button" class="copy-paste-icon" aria-label="Copy max_iter to clipboard" onclick="copyToClipboard('max_iter', this.parentElement.nextElementSibling)" ></button>	max_iter max_iter: int, default=300 Max
<button type="button" class="copy-paste-icon" aria-label="Copy tol to clipboard" onclick="copyToClipboard('tol', this.parentElement.nextElementSibling)" ></button>	tol tol: float, default=1e-4 Relative tolera difference in the clus
<button type="button" class="copy-paste-icon" aria-label="Copy verbose to clipboard" onclick="copyToClipboard('verbose', this.parentElement.nextElementSibling)" ></button>	verbose verbo
<button type="button" class="copy-paste-icon" aria-label="Copy random_state to clipboard" onclick="copyToClipboard('random_state', this.parentElement.nextElementSibling)" ></button>	random_state random_state: int, Randon Determines random number generation make the randomness deterministic
<button type="button" class="copy-paste-icon" aria-label="Copy copy_x to clipboard" onclick="copyToClipboard('copy_x', this.parentElement.nextElementSibling)" ></button>	copy_x copy_x: bool, default=True <br numerically accurate to center the data first. data is not modified. If False, the original da function returns, but small numerical differences then adding the data mean. Note that if the o be made even if copy_x is False. If the ori
<button type="button" class="copy-paste-icon" aria-label="Copy algorithm to clipboard" onclick="copyToClipboard('algorithm', this.parentElement.nextElementSibling)" ></button>	algorithm algorithm: {"lloyd", "elkan"}, default= The classical EM-style algorithm is `"lloyd"`. T on some datasets with well-defined cluster it's more memory intensi shape `(n_samples, n_clusters)`. <br algorithm .. versionchanged:: 1.1<br "auto" and "full". Chan

</details>

<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from the closest toggleable content const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `${paramPrefix}${text}` : text; const originalStyle = element.style; const computedStyle = window.getComputedStyle(element); const originalWidth = computedStyle.width; const originalHTML = element.innerHTML.replace('Copied!', '');

```
navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width = originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false; } document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent = element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected copy-paste icon is missing from the DOM structure'); return; } const paramName = element.parentElement.nextElementSibling .textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `${paramPrefix}${paramName}` : paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a `.features` block; its siblings * contain a `details` element and a table containing feature named. * @returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content = element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` "${row}"`); const formattedText = `[\n${rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ”', ''); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = 'âœ”'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_dat
* @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based
```

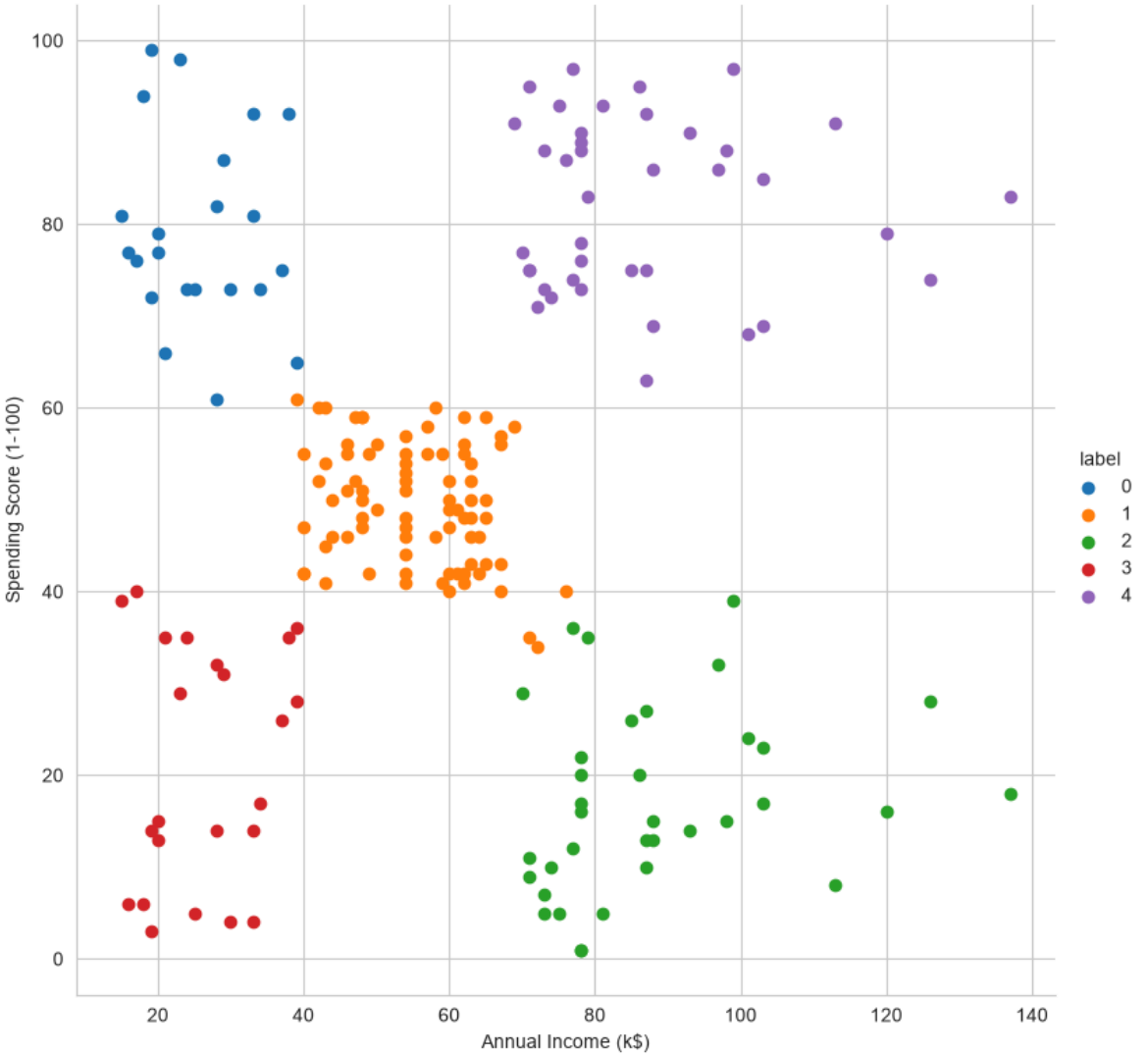
on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s*\(\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*\)\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [5]:
```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Annual Income (k$)</th> <th>Spending Score (1-100)</th> <th>label</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

15	39	3
15	81	0
16	6	3
16	77	0
17	40	3

In [6]:
```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```



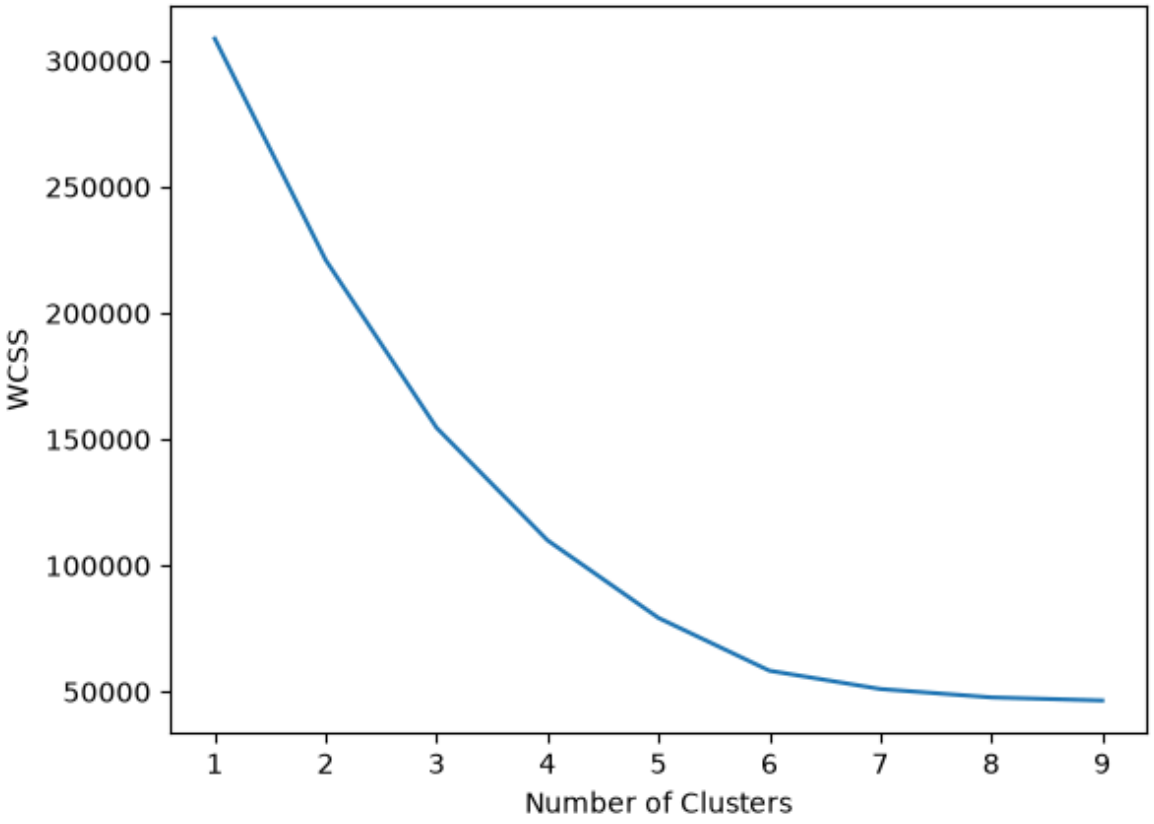
In [5]:
```
features_el = df.iloc[:, [2, 3, 4]].values

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

wcss = []

for i in range(1, 10):
    model = KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)

plt.plot(range(1, 10), wcss)
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```


300000
250000
200000
150000
100000
50000
WCSS
1
2
3
4
5
6
7
8
9
Number of Clusters

In [ ]:

In [4]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\Public\Downloads\Iris dataset.csv")
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

In [5]:
```
df.variety.value_counts()
```

Out[5]:
```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

In [6]:
```
df.head()
```

Out[6]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>sepal.length</th> <th>sepal.width</th> <th>petal.length</th> <th>petal.width</th> <th>variety</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

5.1	3.5	1.4	0.2	Setosa	
4.9	3.0	1.4	0.2	Setosa	
4.7	3.2	1.3	0.2	Setosa	
4.6	3.1	1.5	0.2	Setosa	
5.0	3.6	1.4	0.2	Setosa	

In [8]:
```
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=42
)

model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
```

Out[8]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden] { display: none !important; }` so we also need the `!important` here to be able to override the default hidden behavior on the sphinx rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(--sklearn-color-text-on-default-background)); background-size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-items: center; background-color: var(--sklearn-color-background); padding-right: 1em; padding-left: 1em; } /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the default style - Estimators will overwrite some part of the style using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable { /* Default theme specific background. It is overwritten whether we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-

toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-

link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) { background-color: #f6f6f6; } .estimator-table

.parameters-table tr:hover td { background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104, 0.232); } /* `table td`is set in notebook with right text-align. We need to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell containing the link clickable */ a.param-doc-link::before { position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon { background-image: url(background-repeat: no-repeat; background-size: 14px 14px; background-position: 0; display: inline-block; width: 14px; height: 14px; cursor: pointer; border: none; padding: 0; background-color: transparent; } .copy-paste-icon:focus-visible { outline: 1px solid currentColor; outline-offset: 1px; } .features { font-family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color: var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap: 0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; } .features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before { content: "▸"; color: grey; } .features details[open] > summary .arrow::before { content: "▾"; } .features details:hover > summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features .features-container { max-height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-

color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover { background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style><body>
KNeighborsClassifier()
**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-1" type="checkbox" checked><label for="sk-estimator-id-1" class="sk-toggleable__label fitted sk-toggleable__label-arrow">
KNeighborsClassifier
?Documentation for KNeighborsClassifieriFitted
</label>
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy n_neighbors to clipboard" onclick="copyToClipboard('n_neighbors', this.parentElement.nextElementSibling)" > </button>	n_neighbors n_neighbors: int, default=5<b
<button type="button" class="copy-paste-icon" aria-label="Copy weights to clipboard" onclick="copyToClipboard('weights', this.parentElement.nextElementSibling)" > </button>	weights weights: {'uniform', 'distance'}, Weight function used in prediction. Possib weights. All points in each neighborhood weight points by the inverse of their distance query point will have a greater influence t - [callable] : a user-defined function which returns an array of the same shape conta entitled :ref:`sphx_glr_auto_examples_neighb the impact of the `weights` pa
<button type="button" class="copy-paste-icon" aria-label="Copy algorithm to clipboard" onclick="copyToClipboard('algorithm', this.parentElement.nextElementSibling)" > </button>	algorithm algorithm: {'auto', 'ball_tre Algorithm used to compute the nearest :class:`BallTree` - 'kd_tree' will use :class:`KD search. - 'auto' will attempt to decide the mo the values passed to :meth:`fit` method.<b override the setting o
<button type="button" class="copy-paste-icon" aria-label="Copy leaf_size to	leaf_size leaf_size: int, default=30 Leaf can affect the speed of the

clipboard" onclick="copyToClipboard('leaf_size', this.parentElement.nextElementSibling)" > </button>	memory required to store the tree. The optin
<button type="button" class="copy-paste-icon" aria-label="Copy p to clipboard" onclick="copyToClipboard('p', this.parentElement.nextElementSibling)" > </button>	p p: float, default=2 Power parameter this is equivalent to using manhattan_distanc = 2. For arbitrary p, minkowski_
<button type="button" class="copy-paste-icon" aria-label="Copy metric to clipboard" onclick="copyToClipboard('metric', this.parentElement.nextElementSibling)" > </button>	metric metric: str or callable, default='minkov computation. Default is "minkowski", whic distance when p = 2. See the docun <https://docs.scipy.org/doc/scipy/referen metrics listed in :class:`~sklearn.metr metric values. If metric is "precor matrix and must be square during which case only "nonzero" elements m metric is a callable function, it takes two arrays and must return one value indicating the di works for Scipy's metrics, but is less effic
<button type="button" class="copy-paste-icon" aria-label="Copy metric_params to clipboard" onclick="copyToClipboard('metric_params', this.parentElement.nextElementSibling)" > </button>	metric_params metric_params: dict, def
<button type="button" class="copy-paste-icon" aria-label="Copy n_jobs to clipboard" onclick="copyToClipboard('n_jobs', this.parentElement.nextElementSibling)" > </button>	n_jobs n_jobs: int, default=None <br neighbors search. ``None`` means 1 u context. ``-1`` means using all processors more details

</details>
<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>
<th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

classes_ classes_: array of shape (n_classes,) Class labels known to the classifier	ndarray[object] (3,)	['Setosa','Versicolor','Virginica']
effective_metric_ effective_metric_: str or callble The distance metric used. It will be same as the `metric` parameter or a synonym of it, e.g. 'euclidean' if the `metric` parameter set to 'minkowski' and `p` parameter set to 2.	str	'eu...an'
effective_metric_params_ effective_metric_params_: dict Additional keyword arguments for the metric function. For most metrics will be same with	dict	{}

`metric_params` parameter, but may also contain the `p` parameter value if the `effective_metric_` attribute is set to 'minkowski'.		
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	4
n_samples_fit_ n_samples_fit_: int Number of samples in the fitted data.	int	120
outputs_2d_ outputs_2d_: bool False when `y`'s shape is (n_samples,) or (n_samples, 1) during fit otherwise True.	bool	False

</details>

<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from the closest toggleable content const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `${paramPrefix}${text}` : text; const originalStyle = element.style; const computedStyle = window.getComputedStyle(element); const originalWidth = computedStyle.width; const originalHTML = element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width = originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false; } document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent = element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected copy-paste icon is missing from the DOM structure'); return; } const paramName = element.parentElement.nextElementSibling .textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `${paramPrefix}${paramName}` : paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a `.features` block; its siblings * contain a `details` element and a table containing feature named. * @returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content =

element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` "${row}"`); const formattedText = `[\n${rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ”', ''); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = 'âœ”'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_dat * @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s*\(\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*\)\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [9]:
```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
```

```
0.9666666666666667
1.0
```

In [10]:
```
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
```

Out[10]:
```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])
```

In [11]:
```
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

In [ ]:

In [1]:
```
import numpy as np
from scipy import stats
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
mu_0 = 70
t_stat, p_value = stats.ttest_1samp(marks, mu_0)

print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05

if p_value < alpha:
    print("Reject Null Hypothesis: Mean is significantly different from 7
else:
    print("Fail to Reject Null Hypothesis: No significant difference.")
```

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis: No significant difference.
```

In [ ]:

In [1]:
```
import matplotlib.pyplot as cricket

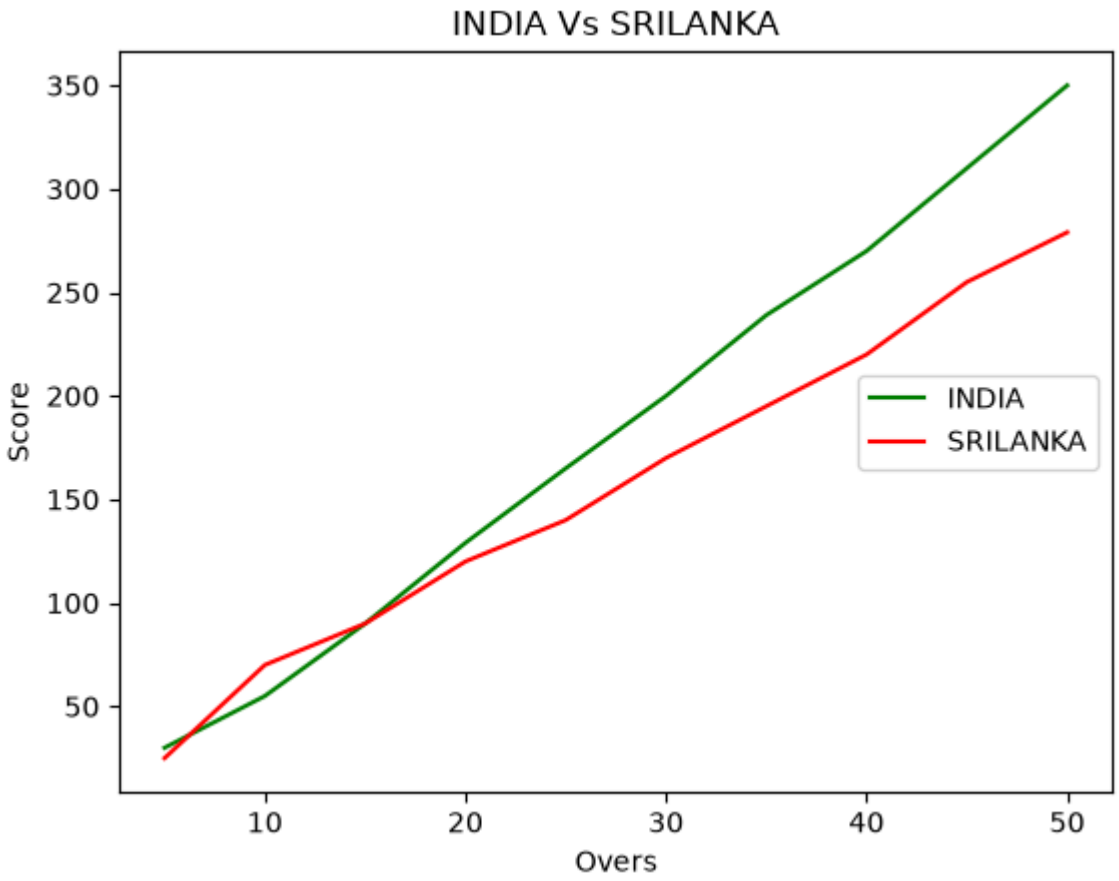
Overs = list(range(5, 51, 5))

Indian_Score = [30, 55, 90, 129, 165, 200, 239, 270, 310, 350]
Srilankan_Score = [25, 70, 90, 120, 140, 170, 195, 220, 255, 279]

cricket.plot(Overs, Indian_Score, color="green", label="INDIA")
cricket.plot(Overs, Srilankan_Score, color="red", label="SRILANKA")

cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")

cricket.legend(loc="center right")
cricket.show()
```



In [2]:
```
import matplotlib.pyplot as hscmark
import numpy as np

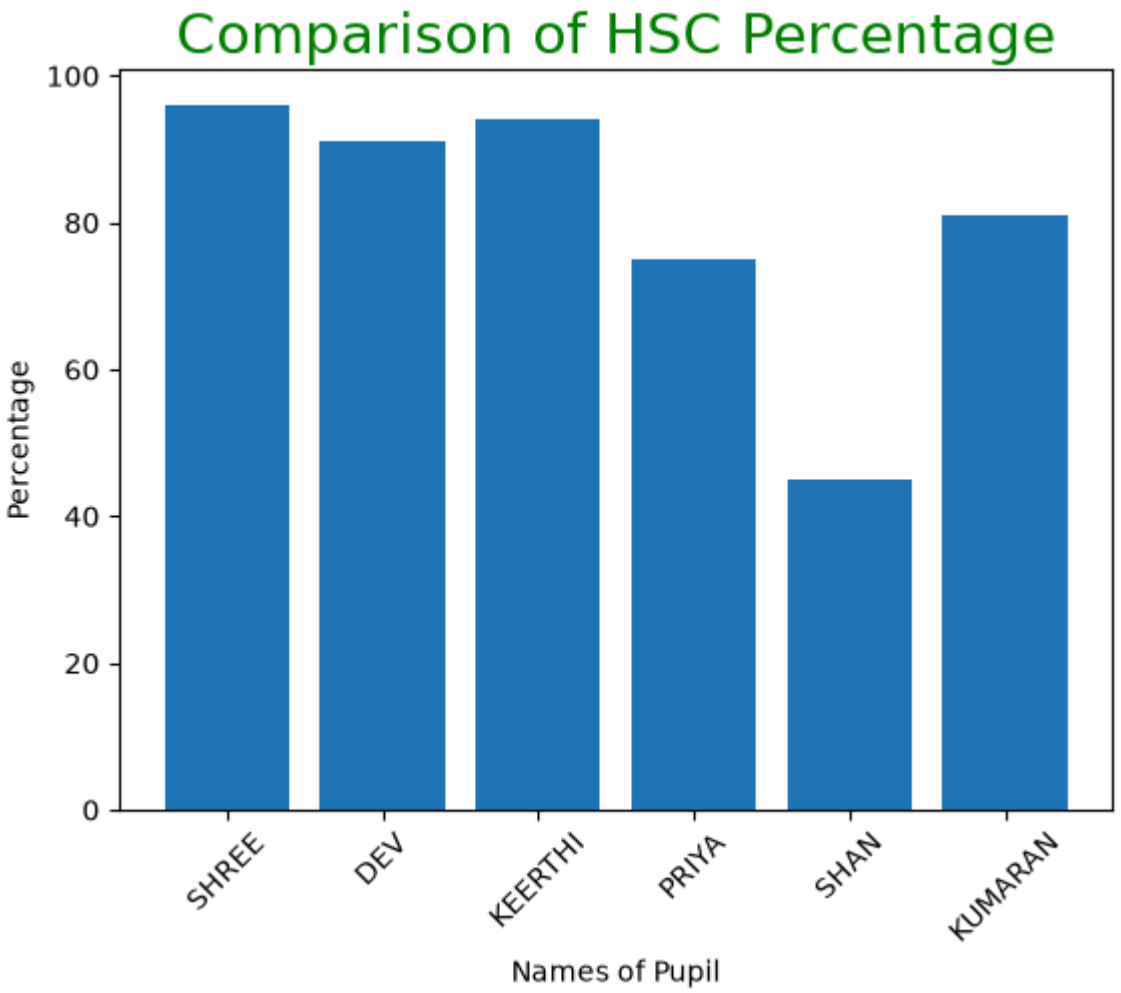
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
Percentage_hsc = [96, 91, 94, 75, 45, 81]

xaxis = np.arange(len(Names))

hscmark.bar(xaxis, Percentage_hsc)

hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
```

```
hscmark.show()
```



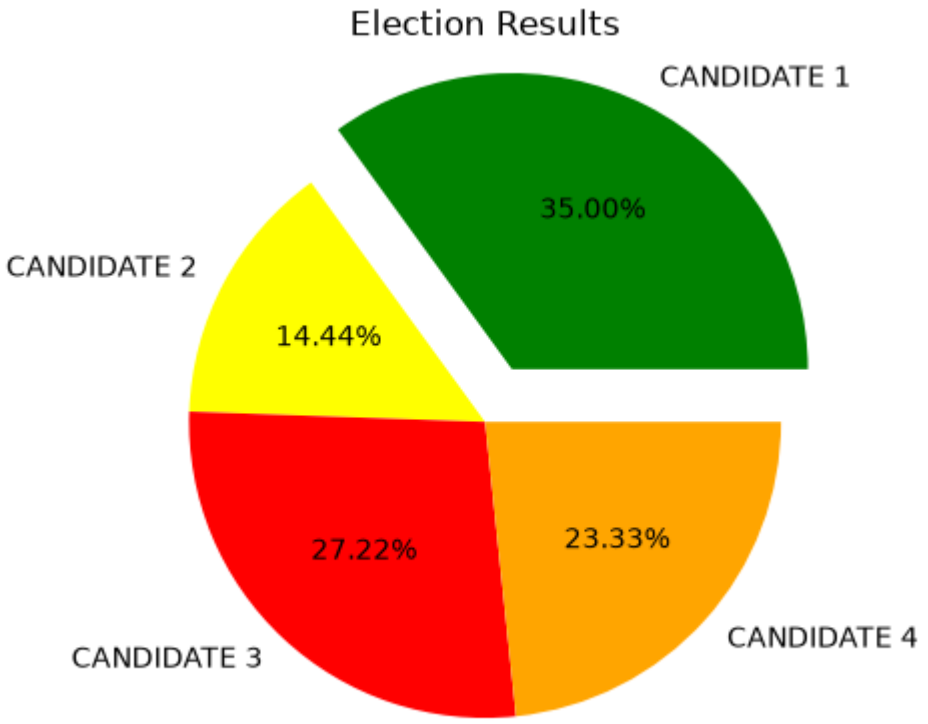
In [3]:
```
import matplotlib.pyplot as election

# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]

colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)

# Plotting the pie chart
election.pie(
    Votes,
    labels=labels,
    colors=colors,
    explode=explode,
    autopct='%0.2f%%'
)

election.title('Election Results')
election.show()
```


Election Results
CANDIDATE 1
35.00%
CANDIDATE 2
14.44%
27.22%
23.33%
CANDIDATE 3
CANDIDATE 4

In [ ]:

In [3]:
```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
from collections import Counter

# Download required datasets
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')

# Load the text
sample = gutenberg.raw("austen-emma.txt")

# Tokenize the text
token = word_tokenize(sample)

# Select the first 50 tokens
wlist = token[:50]

# Calculate word frequencies
wordfreq = Counter(wlist)

# Print word-frequency pairs
print("Pairs")
print([(word, wordfreq[word]) for word in wlist])
```

```
[nltk_data] Downloading package gutenberg to
[nltk_data]     C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\HP\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma',
2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever', 1),
(',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1), ('comf
ortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposition', 1),
(',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1), ('of', 2),
('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('existence', 1),
(';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly', 1), ('twenty-on
e', 1), ('years', 1), ('in', 1), ('the', 2), ('world', 1), ('with', 2)]
```

In [ ]:

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

In [2]:
```
diabetes_df = pd.read_csv("diabetes.csv")

print(diabetes_df.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

In [3]:
```
diabetes_df
```

Out[3]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Pregnancies</th> <th>Glucose</th> <th>BloodPressure</th> <th>SkinThickness</th> <th>Insulin</th> <th>BMI</th> <th>DiabetesPedigreeFunction</th> <th>Age</th> <th>Outcome</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>763</th> <th>764</th> <th>765</th> <th>766</th> <th>767</th> </tbody> <tbody></tbody>

6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...
10	101	76	48	180	32.9	0.171	63	0
2	122	70	27	0	36.8	0.340	27	0
5	121	72	23	112	26.2	0.245	30	0
1	126	60	0	0	30.1	0.349	47	1
1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

In [4]:
```
diabetes_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

In [5]:
```
diabetes_df.describe()
```

Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Pregnancies</th> <th>Glucose</th> <th>BloodPressure</th> <th>SkinThickness</th> <th>Insulin</th> <th>BMI</th> <th>DiabetesPedigreeFunction</th> <th>Age</th> <th>Outcome</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	
3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	
0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	
1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	
3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	
6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	
17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	

In [6]:
```
print("Dataset Shape:", diabetes_df.shape)
```

```
Dataset Shape: (768, 9)
```

In [7]:
```
print(diabetes_df.isnull().sum())
```

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [8]:
```
print("Column Names:")
print(diabetes_df.columns)
```

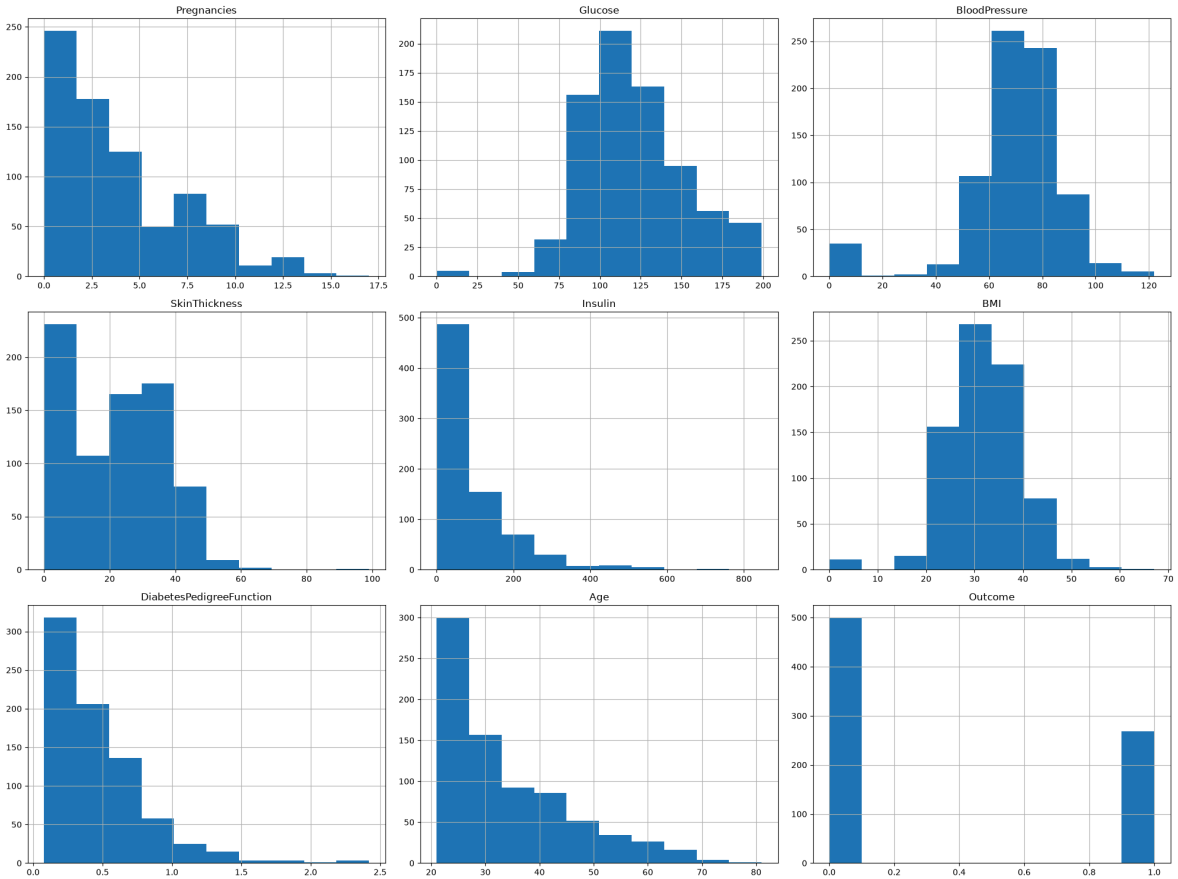
```
Column Names:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insuli
n',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='str')
```

In [9]:
```
print(diabetes_df["Outcome"].value_counts())
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
```

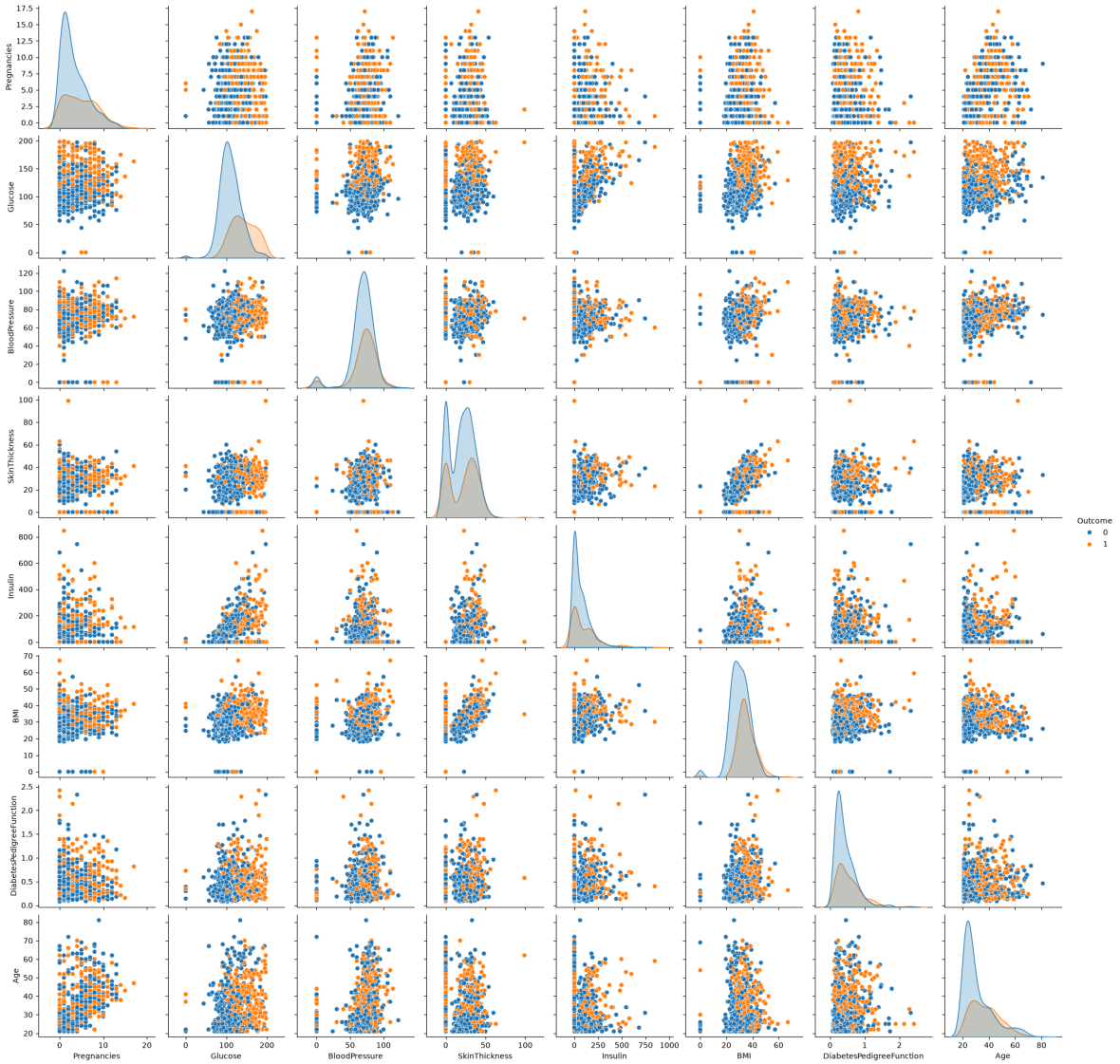
In [10]:
```
diabetes_df.hist(figsize=(20, 15))
```

```
plt.tight_layout()
plt.show()
```



In [11]:
```
sns.pairplot(diabetes_df, hue="Outcome")

plt.show()
```



In [12]:
```
correlation_matrix = diabetes_df.corr(numeric_only=True)

print(correlation_matrix)
```

```
                          Pregnancies   Glucose  BloodPressure  SkinThickn
ess  \
Pregnancies                  1.000000  0.129459       0.141282      -0.081
672
Glucose                      0.129459  1.000000       0.152590       0.057
328
BloodPressure                0.141282  0.152590       1.000000       0.207
371
SkinThickness               -0.081672  0.057328       0.207371       1.000
000
Insulin                     -0.073535  0.331357       0.088933       0.436
783
BMI                          0.017683  0.221071       0.281805       0.392
573
DiabetesPedigreeFunction    -0.033523  0.137337       0.041265       0.183
928
Age                          0.544341  0.263514       0.239528      -0.113
970
Outcome                      0.221898  0.466581       0.065068       0.074
752

                           Insulin       BMI  DiabetesPedigreeFunction  \
Pregnancies              -0.073535  0.017683                 -0.033523
Glucose                   0.331357  0.221071                  0.137337
BloodPressure             0.088933  0.281805                  0.041265
SkinThickness             0.436783  0.392573                  0.183928
Insulin                   1.000000  0.197859                  0.185071
BMI                       0.197859  1.000000                  0.140647
DiabetesPedigreeFunction  0.185071  0.140647                  1.000000
Age                      -0.042163  0.036242                  0.033561
Outcome                   0.130548  0.292695                  0.173844

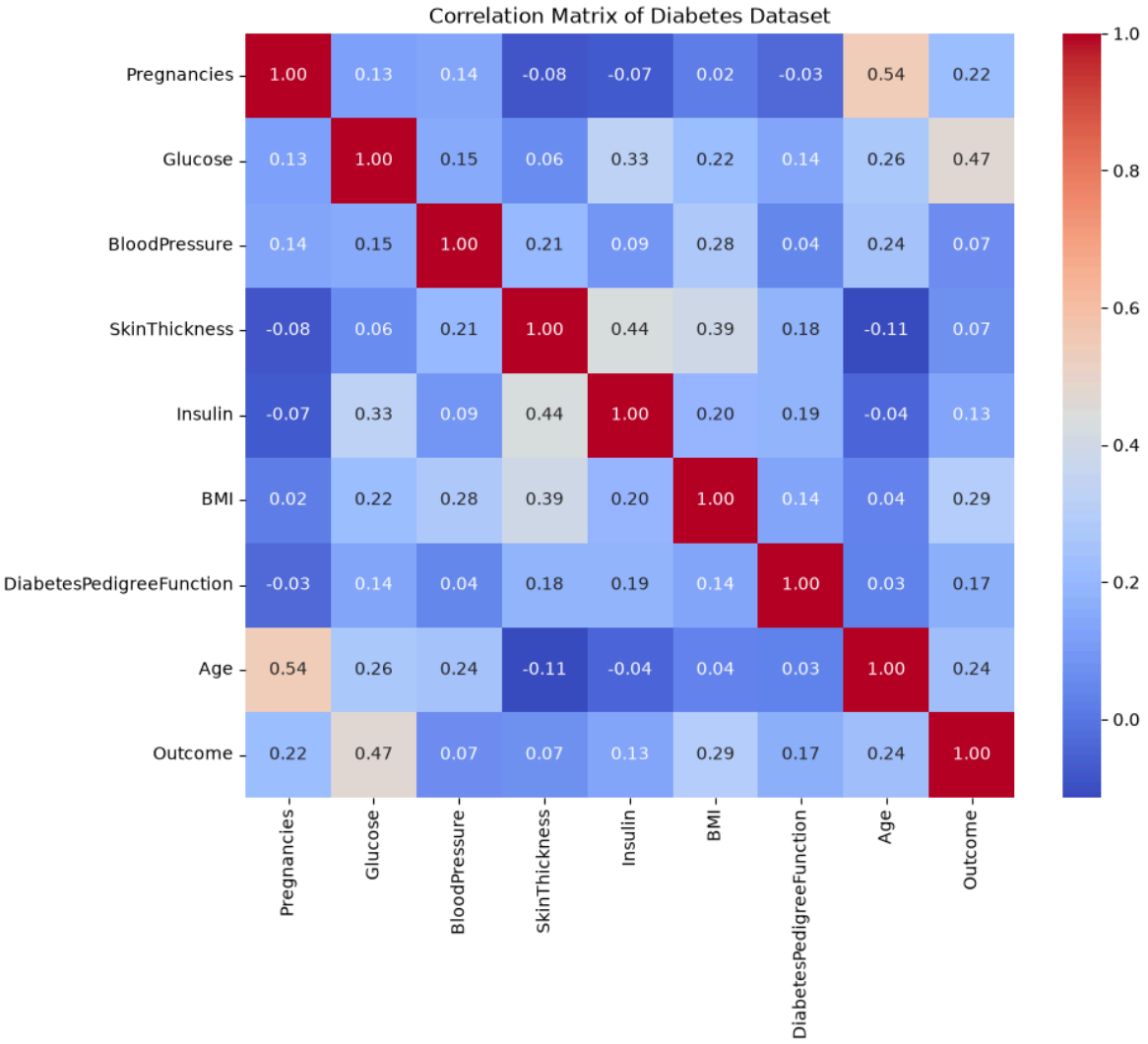
                               Age   Outcome
Pregnancies               0.544341  0.221898
Glucose                   0.263514  0.466581
BloodPressure             0.239528  0.065068
SkinThickness            -0.113970  0.074752
Insulin                  -0.042163  0.130548
BMI                       0.036242  0.292695
DiabetesPedigreeFunction  0.033561  0.173844
Age                       1.000000  0.238356
Outcome                   0.238356  1.000000
```

In [13]:

```
plt.figure(figsize=(10, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix of Diabetes Dataset")
plt.show()
```



In [14]:
```
print(diabetes_df.head(10))
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1
5            5      116             74              0        0  25.6
6            3       78             50             32       88  31.0
7           10      115              0              0        0  35.3
8            2      197             70             45      543  30.5
9            8      125             96              0        0   0.0

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
5                     0.201   30        0
6                     0.248   26        1
7                     0.134   29        0
8                     0.158   53        1
9                     0.232   54        1
```

In [ ]:

In [1]:

```python
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate population
population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(
    population_mean, population_std, population_size
)

# Step 2: Random sampling
sample_sizes = [30, 50, 100]
num_samples = 1000

sample_means = {}

for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

# Step 3: Plot sampling distributions
plt.figure(figsize=(12, 8))

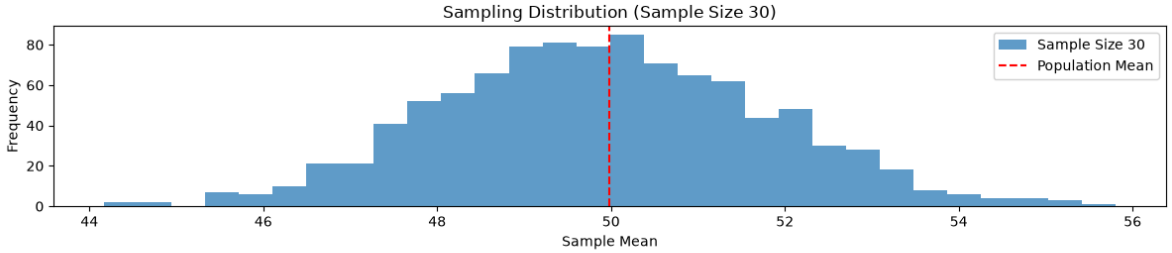
for i, size in enumerate(sample_sizes):
    plt.subplot(3, 1, i + 1)

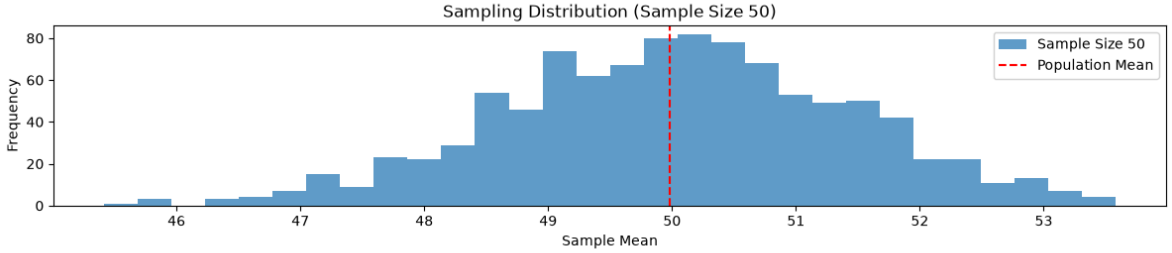
    plt.hist(
        sample_means[size],
        bins=30,
        alpha=0.7,
        label=f'Sample Size {size}'
    )

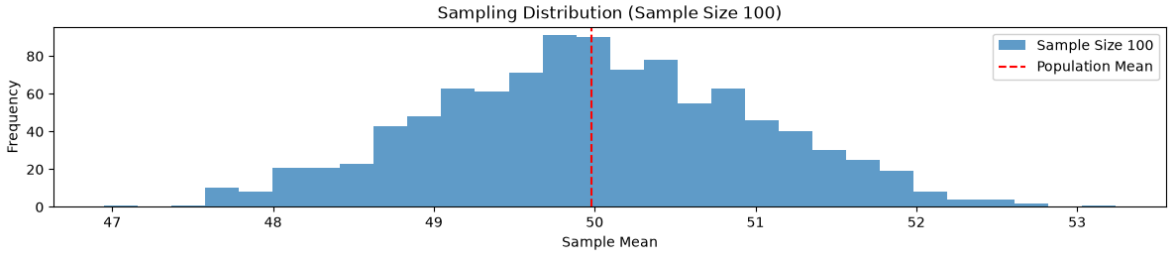
    plt.axvline(
        np.mean(population),
        color='red',
        linestyle='dashed',
        linewidth=1.5,
        label='Population Mean'
    )

    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```


Sampling Distribution (Sample Size 30)
Sample Size 30
Population Mean
Frequency
Sample Mean
0
20
40
60
80
44
46
48
50
52
54
56


Sampling Distribution (Sample Size 50)
Sample Size 50
Population Mean
Frequency
Sample Mean
0
20
40
60
80
46
47
48
49
50
51
52
53


Sampling Distribution (Sample Size 100)
Sample Size 100
Population Mean
Frequency
Sample Mean
0
20
40
60
80
47
48
49
50
51
52
53

In [ ]:

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline
```

In [2]:
```
data = pd.read_csv('Iris_Dataset.csv')
data
```

Out[2]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Id</th> <th>SepalLengthCm</th> <th>SepalWidthCm</th> <th>PetalLengthCm</th> <th>PetalWidthCm</th> <th>variety</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>145</th> <th>146</th> <th>147</th> <th>148</th> <th>149</th> </tbody> <tbody></tbody>

1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...
146	6.7	3.0	5.2	2.3	Iris-virginica
147	6.3	2.5	5.0	1.9	Iris-virginica
148	6.5	3.0	5.2	2.0	Iris-virginica
149	6.2	3.4	5.4	2.3	Iris-virginica
150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

In [3]:
```
data.head()
```

Out[3]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Id</th> <th>SepalLengthCm</th> <th>SepalWidthCm</th> <th>PetalLengthCm</th> <th>PetalWidthCm</th> <th>variety</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5.0	3.6	1.4	0.2	Iris-setosa

In [4]:
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    str
dtypes: float64(4), int64(1), str(1)
memory usage: 7.2 KB
```

In [5]:
```
data.describe()
```

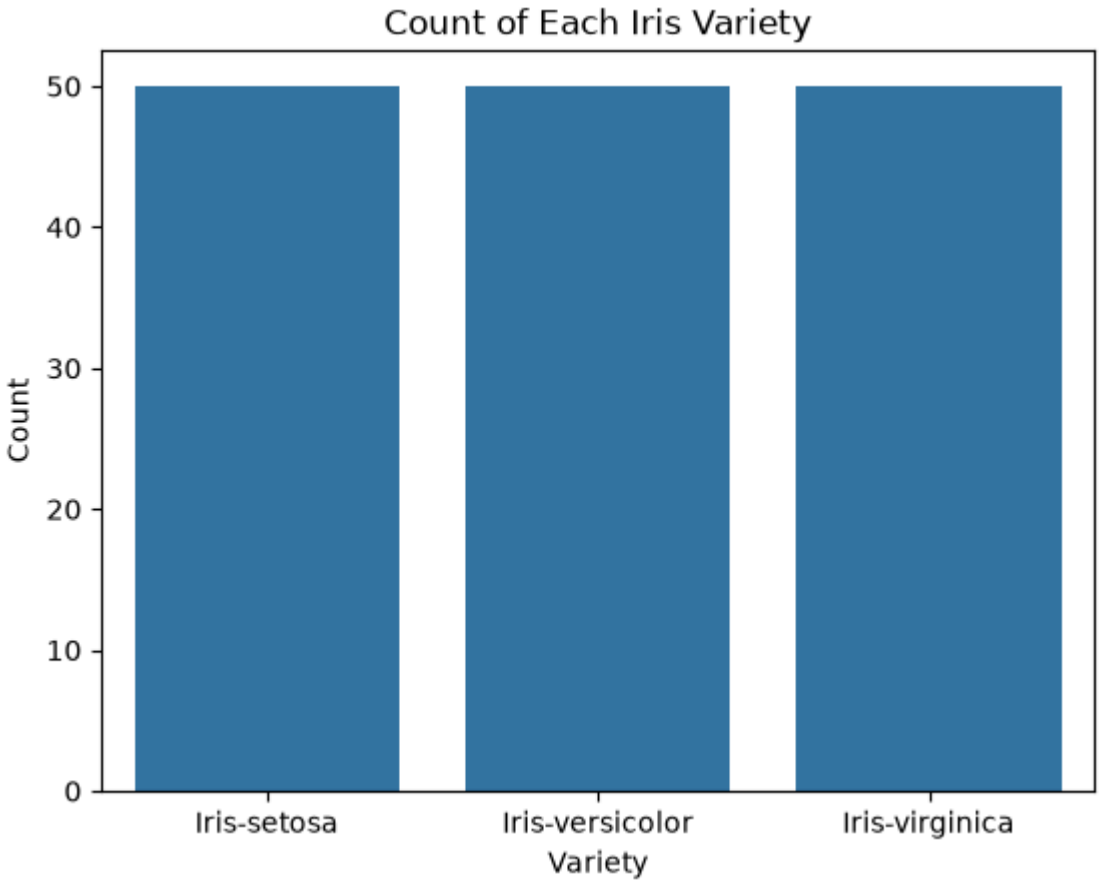
Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Id</th> <th>SepalLengthCm</th> <th>SepalWidthCm</th> <th>PetalLengthCm</th> <th>PetalWidthCm</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

150.000000	150.000000	150.000000	150.000000	150.000000
75.500000	5.843333	3.057333	3.758000	1.199333
43.445368	0.828066	0.435866	1.765298	0.762238
1.000000	4.300000	2.000000	1.000000	0.100000
38.250000	5.100000	2.800000	1.600000	0.300000
75.500000	5.800000	3.000000	4.350000	1.300000
112.750000	6.400000	3.300000	5.100000	1.800000
150.000000	7.900000	4.400000	6.900000	2.500000

In [6]:
```
data.value_counts('variety')
```

Out[6]:
```
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

In [7]:
```
sns.countplot(x='variety', data=data)
plt.title('Count of Each Iris Variety')
plt.xlabel('Variety')
plt.ylabel('Count')
plt.show()
```



In [8]:
```
dummies = pd.get_dummies(data['variety'])
dummies
```

Out[8]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Iris-setosa</th> <th>Iris-versicolor</th> <th>Iris-virginica</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>145</th> <th>146</th> <th>147</th> <th>148</th> <th>149</th> </tbody> <tbody></tbody>

0	True	False	False
1	True	False	False
2	True	False	False
3	True	False	False
4	True	False	False
...	...	...	...
145	False	False	True
146	False	False	True
147	False	False	True
148	False	False	True
149	False	False	True

150 rows × 3 columns

In [9]:

```
FinalDataset = pd.concat(
    [dummies, data.iloc[:, [0, 1, 2, 3]]],
    axis=1
)

FinalDataset.head()
```

Out[9]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Iris-setosa</th> <th>Iris-versicolor</th> <th>Iris-virginica</th> <th>Id</th> <th>SepalLengthCm</th> <th>SepalWidthCm</th> <th>PetalLengthCm</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

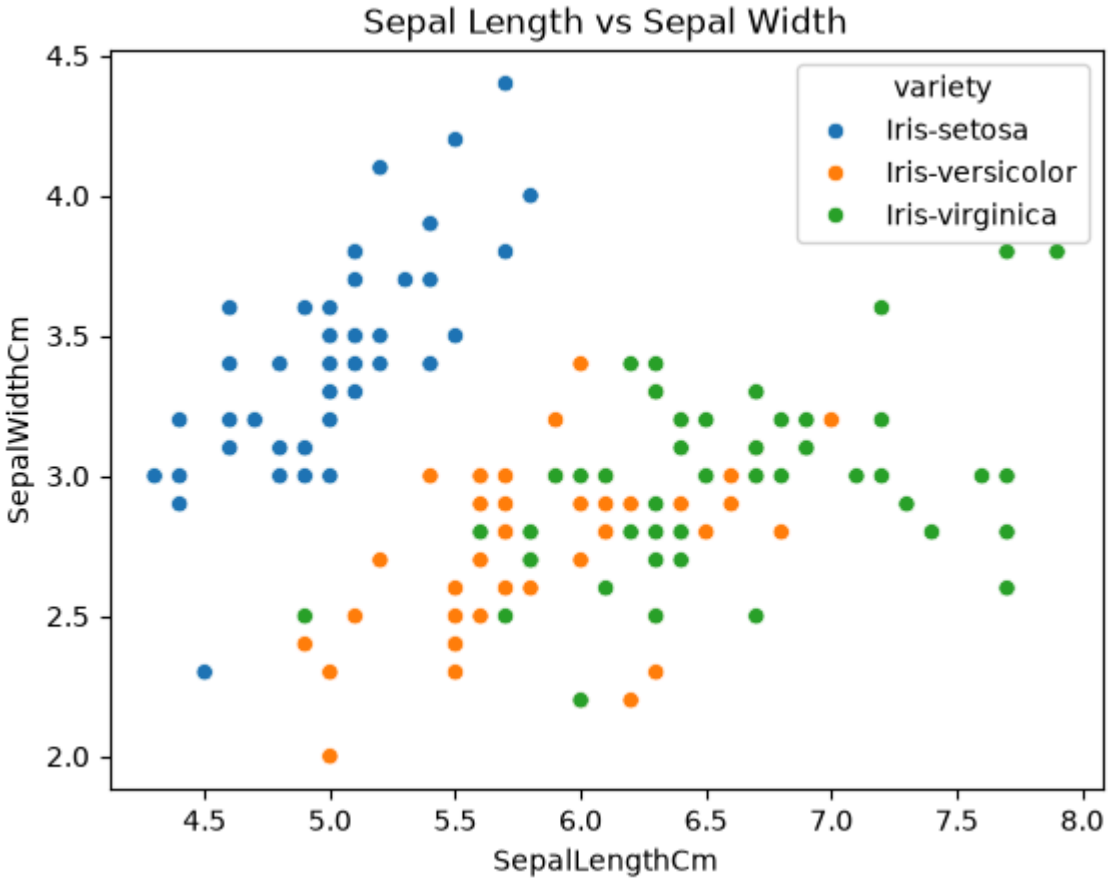
0	True	False	False	1	5.1	3.5	1.4
1	True	False	False	2	4.9	3.0	1.4
2	True	False	False	3	4.7	3.2	1.3
3	True	False	False	4	4.6	3.1	1.5
4	True	False	False	5	5.0	3.6	1.4

In [10]:

```
sns.scatterplot(
    x='SepalLengthCm',
    y='SepalWidthCm',
```

```
    hue='variety',
    data=data
)

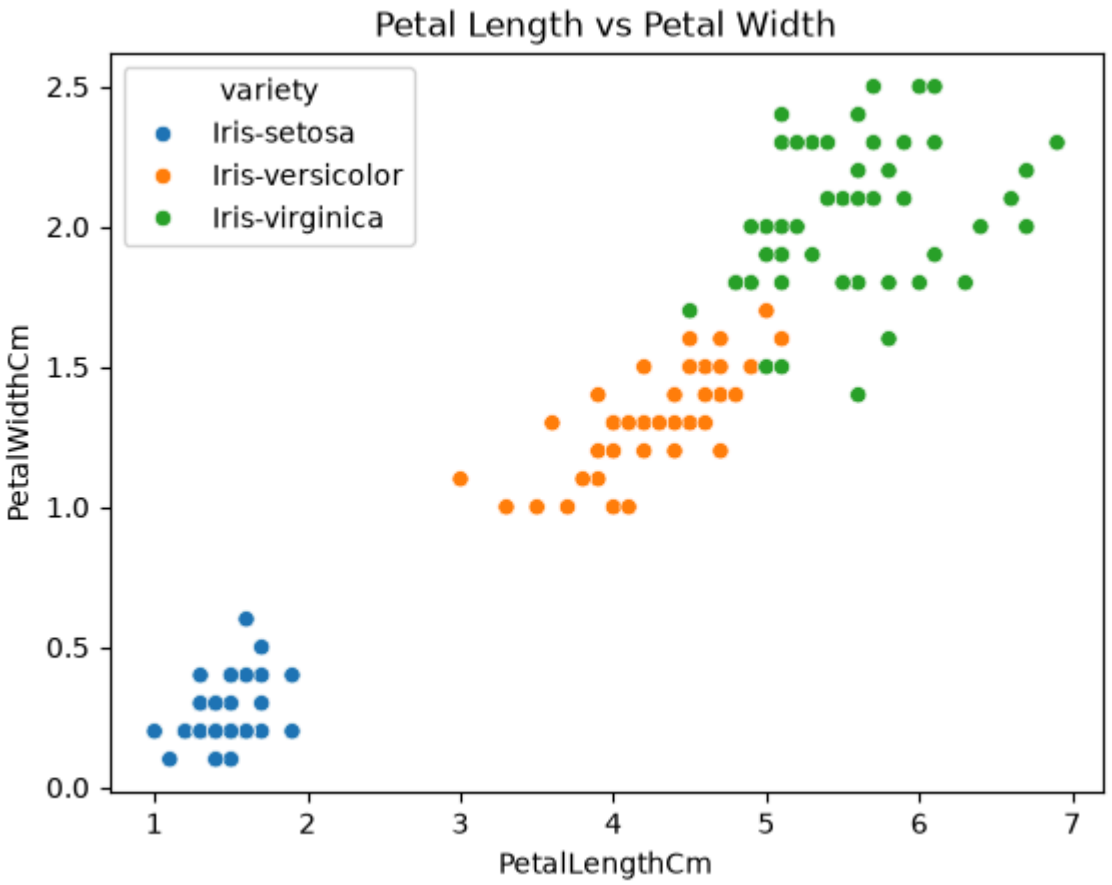
plt.title('Sepal Length vs Sepal Width')
plt.show()
```



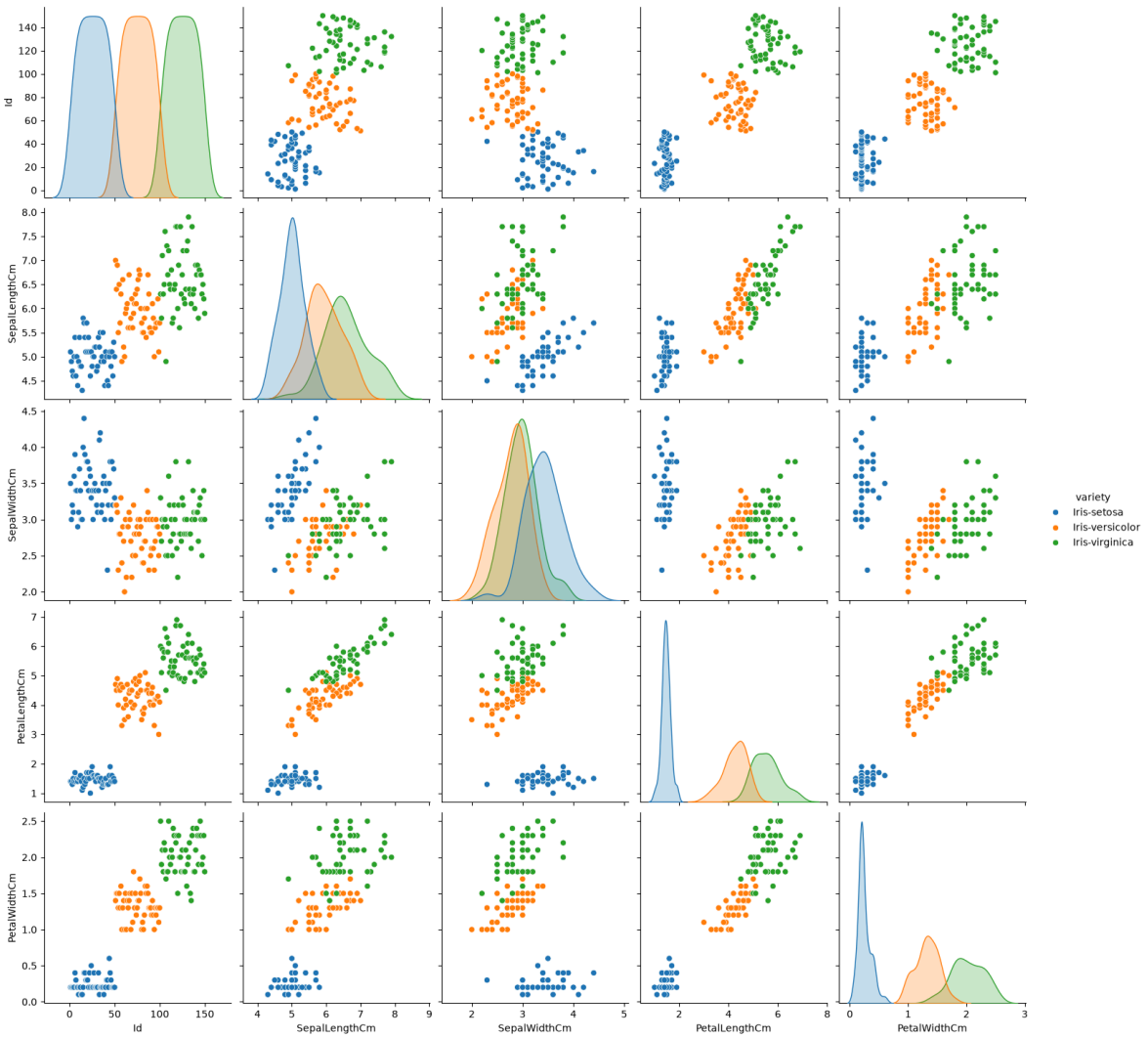
In [11]:

```
sns.scatterplot(
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='variety',
    data=data
)

plt.title('Petal Length vs Petal Width')
plt.show()
```

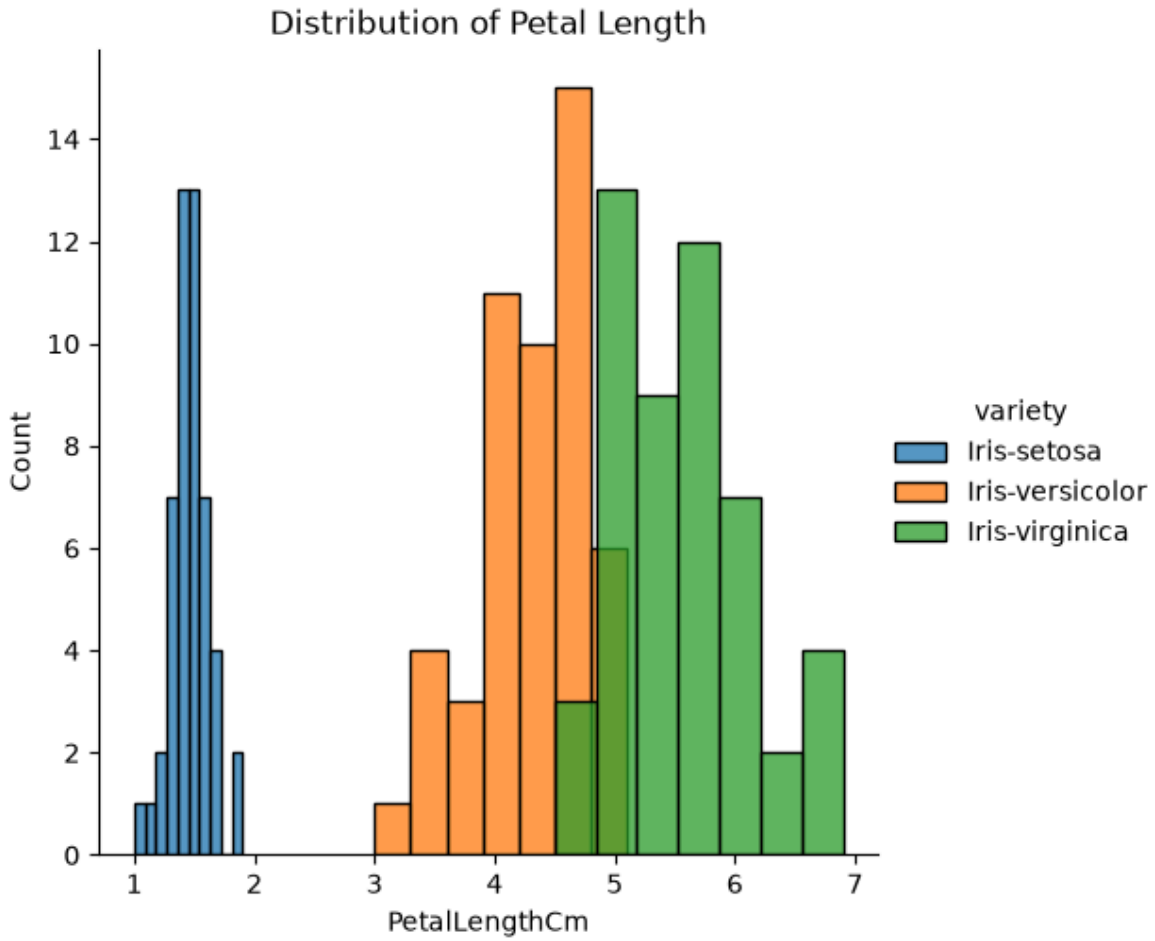


In [12]:
```
sns.pairplot(data, hue='variety', height=3)
plt.show()
```



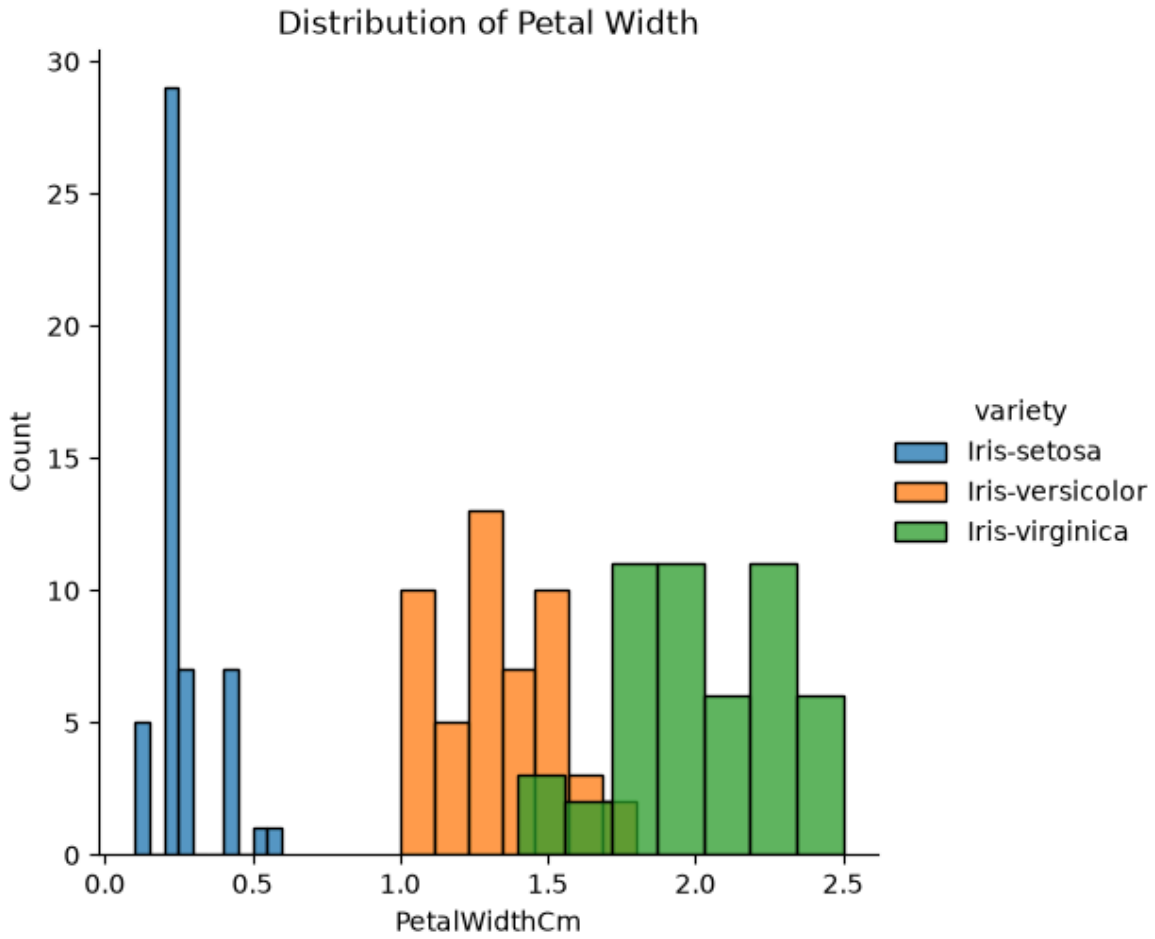
In [13]:
```
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(
    sns.histplot,
    'PetalLengthCm'
).add_legend()

plt.title('Distribution of Petal Length')
plt.show()
```



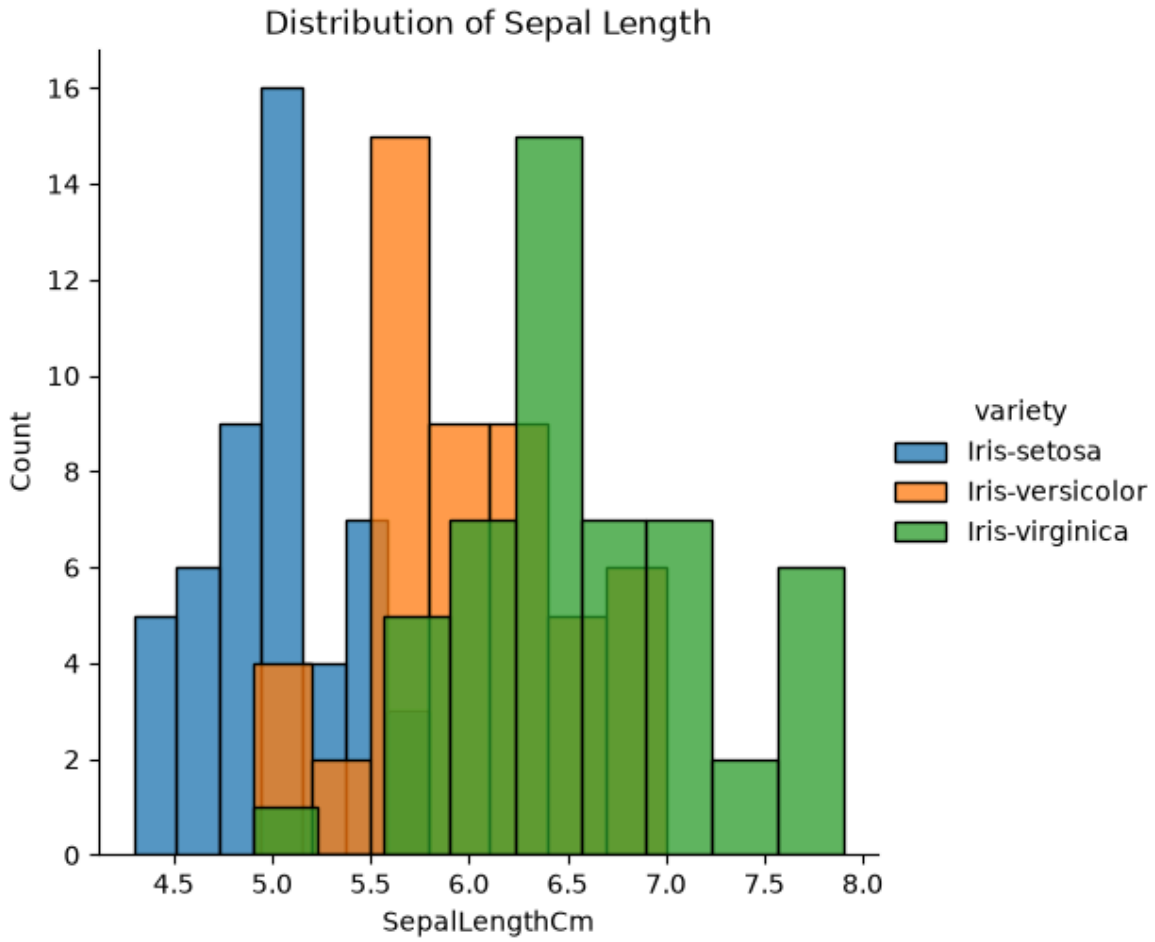
In [14]:
```
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(
    sns.histplot,
    'PetalWidthCm'
).add_legend()

plt.title('Distribution of Petal Width')
plt.show()
```



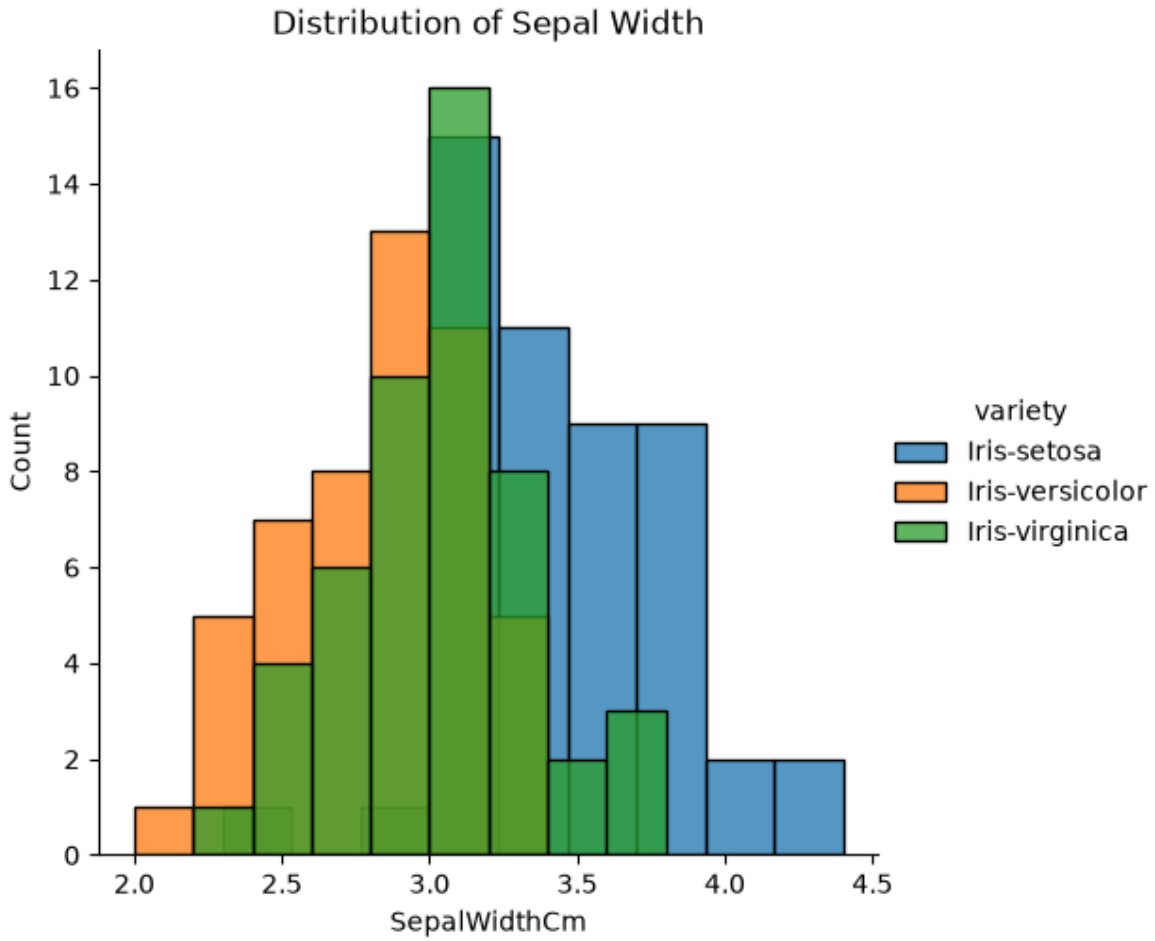
In [15]:
```
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(
    sns.histplot,
    'SepalLengthCm'
).add_legend()

plt.title('Distribution of Sepal Length')
plt.show()
```



In [16]:
```
sns.FacetGrid(
    data,
    hue='variety',
    height=5
).map(
    sns.histplot,
    'SepalWidthCm'
).add_legend()

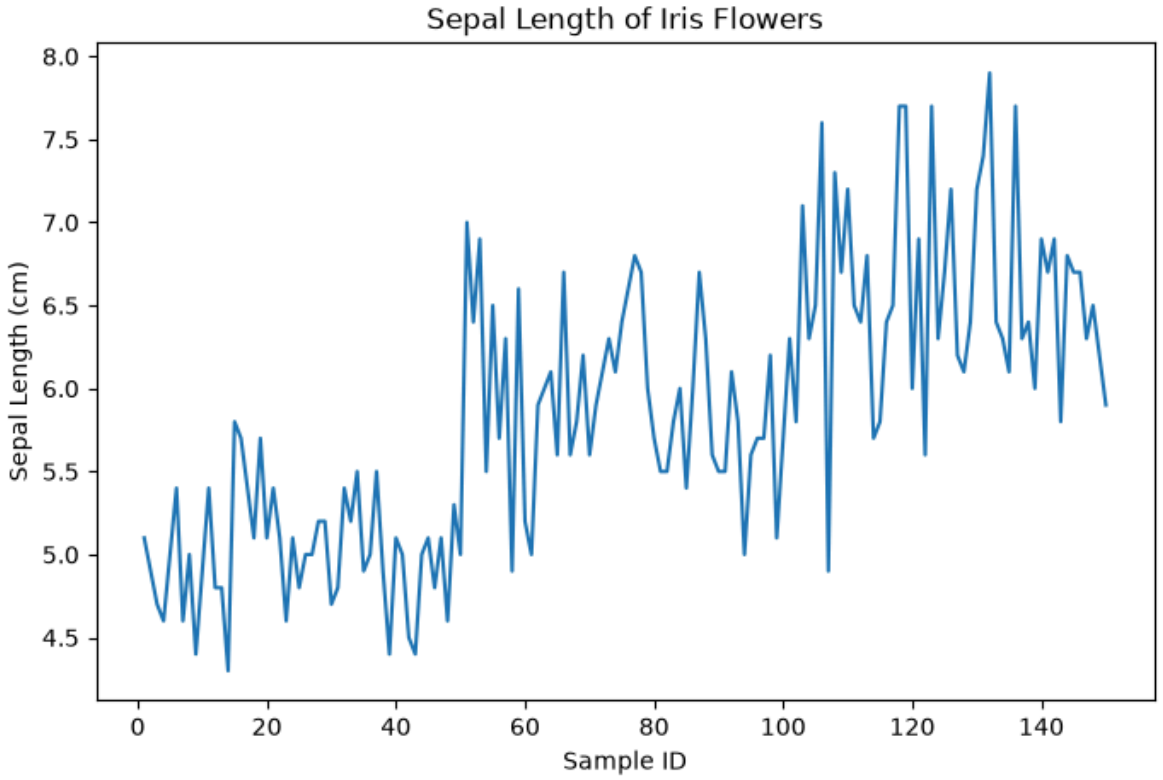
plt.title('Distribution of Sepal Width')
plt.show()
```



In [17]:
```
plt.figure(figsize=(8, 5))

plt.plot(
    data['Id'],
    data['SepalLengthCm']
)

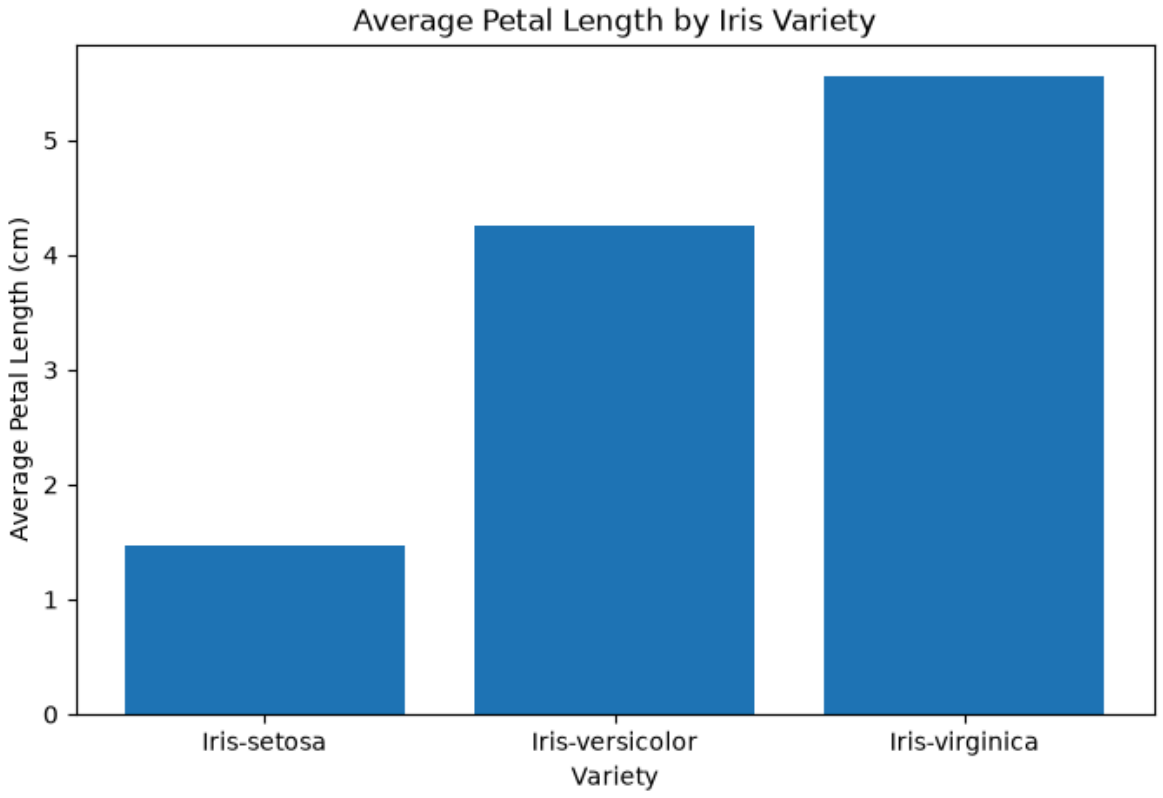
plt.title('Sepal Length of Iris Flowers')
plt.xlabel('Sample ID')
plt.ylabel('Sepal Length (cm)')
plt.show()
```



In [18]:
```
avg_petal = data.groupby('variety')['PetalLengthCm'].mean()

plt.figure(figsize=(8, 5))
plt.bar(avg_petal.index, avg_petal.values)

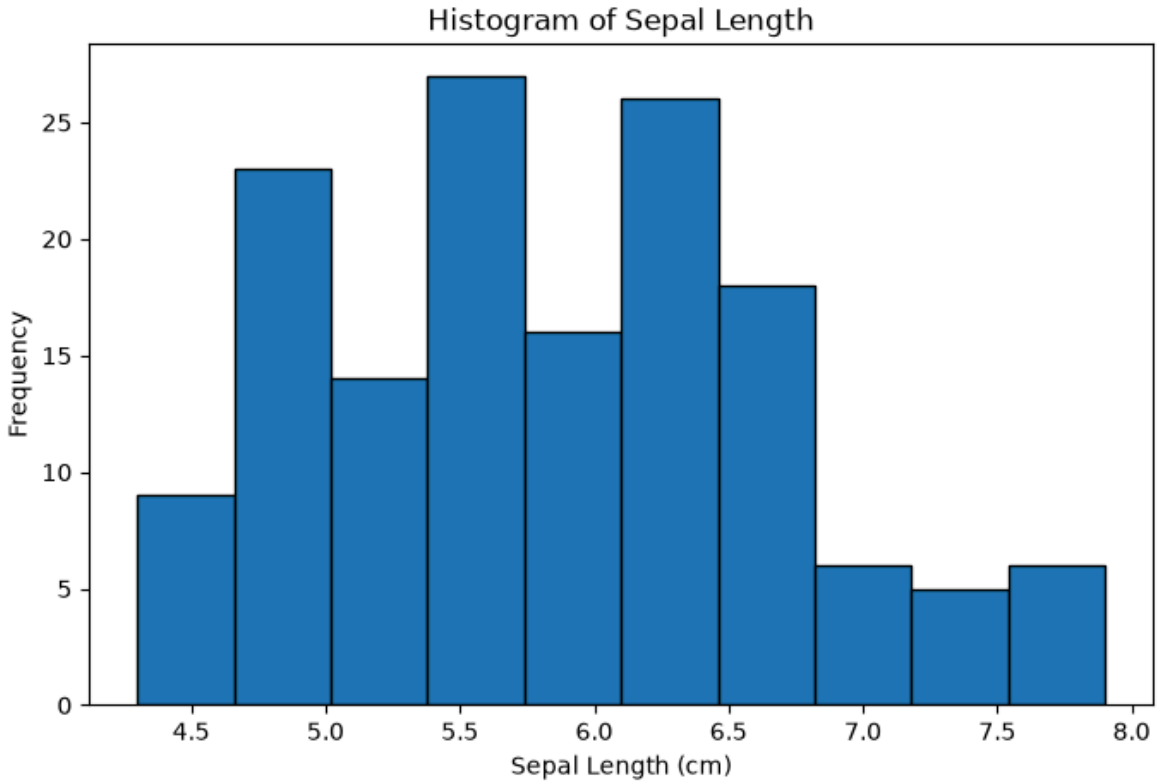
plt.title('Average Petal Length by Iris Variety')
plt.xlabel('Variety')
plt.ylabel('Average Petal Length (cm)')
plt.show()
```



In [19]:
```
plt.figure(figsize=(8, 5))

plt.hist(data['SepalLengthCm'], bins=10, edgecolor='black')

plt.title('Histogram of Sepal Length')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Frequency')
plt.show()
```

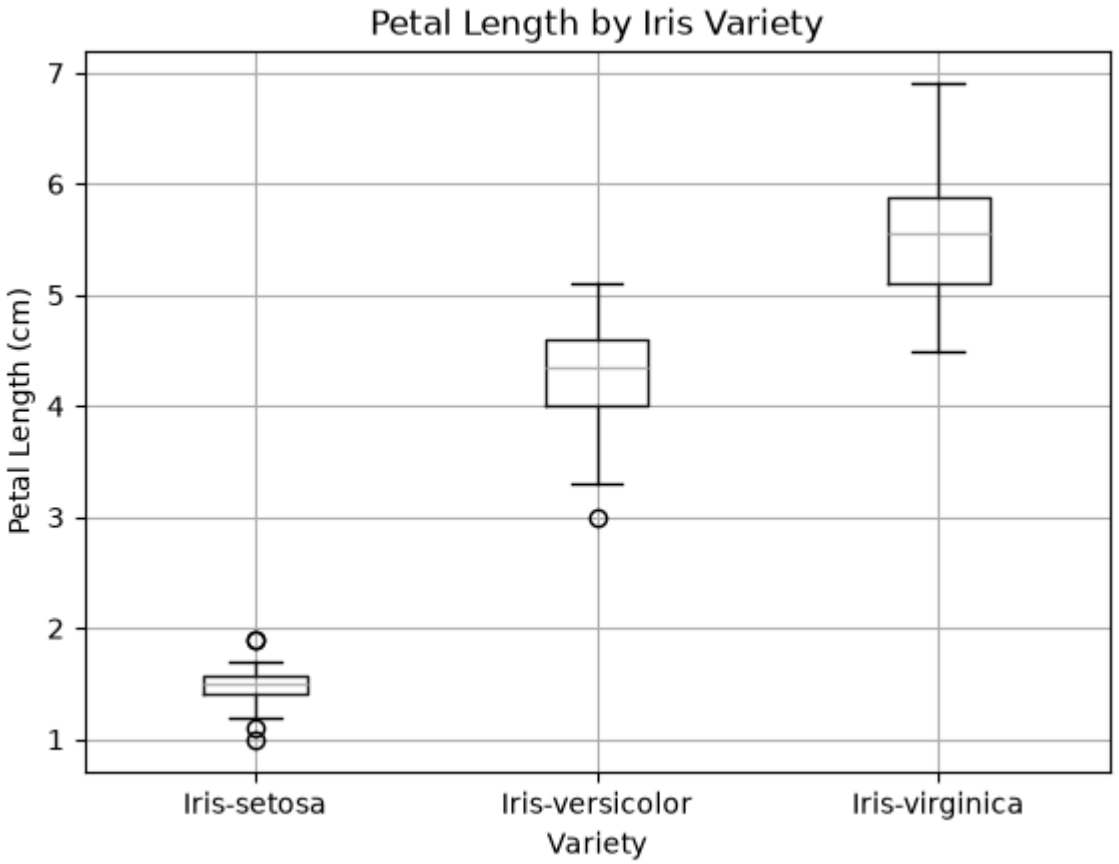


In [20]:
```
plt.figure(figsize=(8, 5))

data.boxplot(column='PetalLengthCm', by='variety')

plt.title('Petal Length by Iris Variety')
plt.suptitle('')
plt.xlabel('Variety')
plt.ylabel('Petal Length (cm)')
plt.show()
```

```
<Figure size 800x500 with 0 Axes>
```


Petal Length by Iris Variety
Petal Length (cm)
7
6
5
4
3
2
1
Iris-setosa
Iris-versicolor
Iris-virginica
Variety

In [ ]:

In [2]:

```
import numpy as np

# Create a random array
array = np.random.randint(1, 100, 9)
print("Original Array:")
print(array)

# Square root of array elements
print("\nSquare Root:")
print(np.sqrt(array))

# Check dimensions
print("\nNumber of Dimensions:")
print(array.ndim)

# Reshape into 3x3 matrix
new_array = array.reshape(3, 3)
print("\nReshaped Array:")
print(new_array)

# Check dimensions of 2D array
print("\nDimensions of New Array:")
print(new_array.ndim)

# Convert 2D array into 1D array
print("\nRavel Array:")
print(new_array.ravel())

# Reshape again
newm = new_array.reshape(3, 3)
print("\nNew Matrix:")
print(newm)

# Array slicing
print("\nnewm[2, 1:3]:")
print(newm[2, 1:3])

print("\nnewm[1:2, 1:3]:")
print(newm[1:2, 1:3])

print("\nnew_array[0:3, 0:0]:")
print(new_array[0:3, 0:0])

print("\nnew_array[0:2, 0:1]:")
print(new_array[0:2, 0:1])

print("\nnew_array[0:3, 0:1]:")
print(new_array[0:3, 0:1])

print("\nnew_array[1:3]:")
print(new_array[1:3])
```

```
Original Array:
[32 12 27 32 30 45 48 10 52]

Square Root:
[5.65685425 3.46410162 5.19615242 5.65685425 5.47722558 6.70820393
 6.92820323 3.16227766 7.21110255]

Number of Dimensions:
1

Reshaped Array:
[[32 12 27]
 [32 30 45]
 [48 10 52]]

Dimensions of New Array:
2

Ravel Array:
[32 12 27 32 30 45 48 10 52]

New Matrix:
[[32 12 27]
 [32 30 45]
 [48 10 52]]

newm[2, 1:3]:
[10 52]

newm[1:2, 1:3]:
[[30 45]]

new_array[0:3, 0:0]:
[]

new_array[0:2, 0:1]:
[[32]
 [32]]

new_array[0:3, 0:1]:
[[32]
 [32]
 [48]]

new_array[1:3]:
[[32 30 45]
 [48 10 52]]
```

In [ ]:

In [1]:

```python
import numpy as np
import pandas as pd

# Create a DataFrame using a list
data = [[1, 'Smith', 50000],
        [2, 'Jones', 60000]]

df = pd.DataFrame(data)
print("Initial DataFrame:")
print(df)

# Assign column names
df.columns = ['Empd', 'Name', 'Salary']
print("\nDataFrame with Column Names:")
print(df)

# Display DataFrame information
print("\nDataFrame Information:")
df.info()

# Create employee dataset
employee_data = {
    'emp id': [1, 2, 3, 4, 5, 6, 7],
    'name': ['SREE VARSSINI K S', 'SREEMATHI B', 'SREYA G',
             'SREYASKARI MULLAPUDI', 'SRI AKASH U G',
             'SRI HARSHAVARDHANAN R', 'SRI HARSHAVARDHANAN R'],
    'salary': [5000, 6000, 7000, 5000, 8000, 3000, 6000]
}

df = pd.DataFrame(employee_data)

print("\nEmployee DataFrame:")
print(df)

# Display first five rows
print("\nFirst Five Rows:")
print(df.head())

# Display last five rows
print("\nLast Five Rows:")
print(df.tail())

# Display information
print("\nEmployee DataFrame Information:")
df.info()

# Select salary column
print("\nSalary Column:")
print(df['salary'])

# Calculate statistics
print("\nMean Salary:", df['salary'].mean())
print("Median Salary:", df['salary'].median())
print("Mode Salary:")
print(df['salary'].mode())
print("Salary Variance:", df['salary'].var())
print("Salary Standard Deviation:", df['salary'].std())
```

```python
# Statistical summary
print("\nStatistical Summary:")
print(df.describe())

print("\nComplete Statistical Summary:")
print(df.describe(include='all'))

# Display column names
empCol = df.columns
print("\nColumn Names:")
print(empCol)

# Convert DataFrame into NumPy array
emparray = df.values
print("\nNumPy Array:")
print(emparray)

# Create DataFrame from NumPy array
employee_DF = pd.DataFrame(emparray, columns=empCol)
print("\nDataFrame Created from NumPy Array:")
print(employee_DF)
```

```
Initial DataFrame:
   0      1      2
0  1  Smith  50000
1  2  Jones  60000

DataFrame with Column Names:
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 180.0 bytes

Employee DataFrame:
   emp id                    name  salary
0       1      SREE VARSSINI K S    5000
1       2             SREEMATHI B    6000
2       3                 SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5          SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000

First Five Rows:
   emp id                   name  salary
0       1     SREE VARSSINI K S    5000
1       2            SREEMATHI B    6000
2       3                SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000

Last Five Rows:
   emp id                    name  salary
2       3                 SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5          SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000

Employee DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 300.0 bytes
```

```
Salary Column:
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64

Mean Salary: 5714.285714285715
Median Salary: 6000.0
Mode Salary:
0    5000
1    6000
Name: salary, dtype: int64
Salary Variance: 2571428.5714285714
Salary Standard Deviation: 1603.5674514745463

Statistical Summary:
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000

Complete Statistical Summary:
          emp id                    name       salary
count   7.000000                       7     7.000000
unique       NaN                       6          NaN
top          NaN  SRI HARSHAVARDHANAN R          NaN
freq         NaN                       2          NaN
mean    4.000000                     NaN  5714.285714
std     2.160247                     NaN  1603.567451
min     1.000000                     NaN  3000.000000
25%     2.500000                     NaN  5000.000000
50%     4.000000                     NaN  6000.000000
75%     5.500000                     NaN  6500.000000
max     7.000000                     NaN  8000.000000

Column Names:
Index(['emp id', 'name', 'salary'], dtype='str')

NumPy Array:
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHANAN R' 3000]
 [7 'SRI HARSHAVARDHANAN R' 6000]]

DataFrame Created from NumPy Array:
  emp id                   name salary
0      1      SREE VARSSINI K S   5000
1      2            SREEMATHI B   6000
```

```
2      3               SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5          SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

In [ ]: